

The University of Manchester
Department of Computer Science
Project Report 2025

**Evaluating Deep Learning Acceleration on FPGA:
NVDLA Case Study**

Author: Jacob Uday Georgis

Supervisor: Professor Davide Bertozzi
Co-supervisor: Alessandro Veronesi

Word count: 9848

Abstract

Evaluating Deep Learning Acceleration on FPGA: NVDLA Case Study

Author: Jacob Uday Georgis

Deep neural networks have become increasingly adopted to perform a variety of tasks in a range of domains. To meet this increase in demand, there has been widespread interest in investigating architectures that are specially purposed towards accelerating these tasks. One such accelerator is the NVIDIA Deep Learning Accelerator (NVDLA), an open-sourced accelerator design that offers scalability, efficiency, and a software ecosystem for end-to-end model deployment. With the reconfigurability of FPGAs naturally lending themselves towards prototyping of open-sourced hardware, part of this project focuses on implementing NVDLA on FPGA. The integration of the accelerator then allowed for exploration into performing convolution operations standalone, allowing developers to deploy deep learning models without the use of the software stack.

In this project, we attempted to configure the accelerator to perform a direct convolution bare-metal. Although the resultant flexibility of this interface falls short due to errors, quantifying the results of performing valid convolution procedures on NVDLA against a single-core ARM processor shows that the accelerator consistently leads to performance gains. This implementation provides a stepping-stone towards future work with the accelerator, allowing the hardware to become accessible without the software ecosystem.

Supervisor: Professor Davide Bertozzi

Co-supervisor: Alessandro Veronesi

Acknowledgements

I would like to express my deep gratitude to my supervisors, Davide Bertozzi and Alessandro Veronesi, for their invaluable help throughout this project. I truly could not have done this project without their support. I would also like to thank my friends and family for supporting me during such a challenging year.

Declaration

No portion of the work referred to in this report has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

Copyright

- i. The author of this thesis (including any appendices and/or schedules to this thesis) owns certain copyright or related rights in it (the “Copyright”) and s/he has given The University of Manchester certain rights to use such Copyright, including for administrative purposes.
- ii. Copies of this thesis, either in full or in extracts and whether in hard or electronic copy, may be made **only** in accordance with the Copyright, Designs and Patents Act 1988 (as amended) and regulations issued under it or, where appropriate, in accordance with licensing agreements which the University has from time to time. This page must form part of any such copies made.
- iii. The ownership of certain Copyright, patents, designs, trade marks and other intellectual property (the “Intellectual Property”) and any reproductions of copyright works in the thesis, for example graphs and tables (“Reproductions”), which may be described in this thesis, may not be owned by the author and may be owned by third parties. Such Intellectual Property and Reproductions cannot and must not be made available for use without the prior written permission of the owner(s) of the relevant Intellectual Property and/or Reproductions.
- iv. Further information on the conditions under which disclosure, publication and commercialisation of this thesis, the Copyright and any Intellectual Property and/or Reproductions described in it may take place is available in the University IP Policy (see <http://documents.manchester.ac.uk/DocuInfo.aspx?DocID=24420>), in any relevant Thesis restriction declarations deposited in the University Library, The University Library’s regulations (see <http://www.library.manchester.ac.uk/about/regulations/>) and in The University’s policy on presentation of Theses

Contents

1	Introduction	8
1.1	Motivation	8
1.2	Aims	10
1.3	Objectives	10
1.4	Success Criteria	10
1.5	Structure	11
2	Background	12
2.1	Convolutional Neural Networks	12
2.1.1	Artificial Neural Networks	12
2.1.2	Convolution	15
2.1.3	Pooling	16
2.1.4	Architecture	17
2.2	Field-Programmable Gate Array	18
2.3	Deep Learning Acceleration	18
2.3.1	Design Objectives	18
2.3.2	Design Strategies	19
3	NVIDIA Deep Learning Accelerator	24
3.1	Repository Overview	24
3.2	Hardware	25
3.2.1	Core Overview	25
3.2.2	Convolution Pipeline	26
3.2.3	Single Point Data Processor	29
3.2.4	Feature Data Format	30
3.2.5	Integration	32
3.3	Software ecosystem	35
3.3.1	Compiler	35

3.3.2	Runtime Environment	35
3.4	Related Work	36
4	Methodology	37
4.1	Hardware	37
4.1.1	FPGA Hardware Overview	37
4.1.2	Vivado Implementation Details	38
4.2	Software	42
4.2.1	Vitis Implementation Details	42
4.2.2	Tensor Library	43
4.2.3	Software Macros	46
4.2.4	Sanity Testing	46
4.2.5	NVDLA Direct Convolution	48
5	Results and Evaluation	51
5.1	FPGA Implementation Statistics	51
5.1.1	Resource Utilisation Analysis	51
5.1.2	Timing Summary	53
5.2	Hardware against Software Convolution	54
5.2.1	Errors	54
5.2.2	Fully Connected Layer Timings	54
5.2.3	Convolution Timings	55
6	Conclusion	56
6.1	Summary of Achievements	56
6.2	Critical Reflection	57
6.3	Ideas for Future Work	57
	Bibliography	58
A	Example of operation	62
A.1	Xilinx NVDLA Wrapper	62
A.2	Tensor Reshape Functions	66
A.3	NVDLA Register Offsets	67
A.4	Stage SDP Passthrough NVDLA	75
A.5	Stage Direct Convolution NVDLA	78
A.6	Stage Direct Convolution NVDLA	82

Abbreviations

NVDLA	NVIDIA Deep Learning Accelerator
DLA	Deep Learning Accelerator
FPGA	Field-Programmable Gate Array
ASIC	Application-Specific Integrated Circuit
ML	Machine Learning
DL	Deep Learning
CNN	Convolutional Neural Networks
ANN	Artificial Neural Networks
DNN	Deep Neural Networks
SLP	Single-Layer Perceptron
MLP	Multi-Layer Perceptron
CPU	Central Processing Unit
GPU	Graphics Processing Unit
TPU	Tensor Processing Unit
CLB	Configurable Logic Block
LUT	Lookup Table
MAC	Multiply-Accumulate
ALU	Arithmetic Logic Unit
PE	Processing Element
RTL	Register-Transfer Level
DSP	Digital Signal Processor
FP	Floating Point
SDP	Single Data Processor
PDP	Planar Data Processor

CDP	Cross-Channel Data Processor
RUBIK	Data Reshape Engine
DMA	Direct Memory Access
BDMA	Bridge DMA
(R/W)DMA	(Read/Write) DMA
CDMA	Convolution DMA
CBUF	Convolution Buffer
CSC	Convolution Sequence Controller
CACC	Convolution Accumulator
DC	Direct Convolution
SoC	System on Chip
CSB	Configuration Space Bus
PL	Programmable Logic
PS	Processing System
APU	Application Processing Unit
ILA	Integrated Logic Analyser
BSP	Board Support Package

Chapter 1

Introduction

1.1 Motivation

The past decade has seen immense development in the field of deep learning (DL). Deep learning is defined as a subset of machine learning that aims to simulate human-like decision-making by performing several layers of complex computation [1]. This development powers many mainstream applications we see and hear about today, such as self-driving cars, large-language models, and facial recognition. As the complexity of these models increases, so does the amount of compute required to utilise them, as illustrated in Figure 1.1.

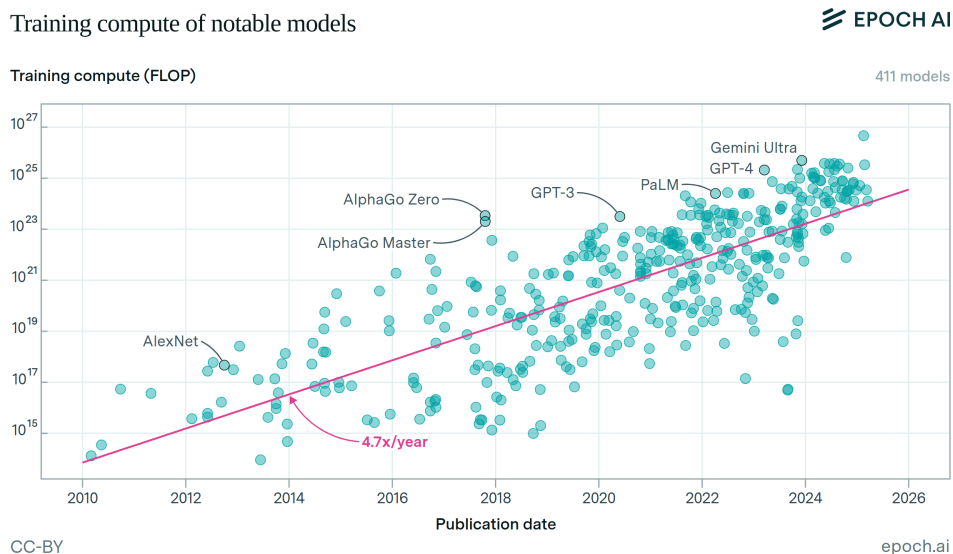


Figure 1.1: A graph of training compute in floating point operations (FLOPs) against publication date, showing that the training compute used to train models is doubling roughly every 5 months. Source: [2].

Despite excelling at general-purpose tasks, modern central processing units (CPUs) exhibit architectural bottlenecks that hinder their performance in DL tasks. A key issue is their parallel processing capability. A popular alternative to the CPU is the graphics processing unit (GPU), which is designed to accommodate highly parallelizable tasks. Despite this, the versatility in the GPU architecture, built to handle a broad range of parallel tasks, can lead to inefficiencies in power consumption for DL tasks [3]. With applications of DL becoming more pervasive, there is a need for specialised hardware coprocessors that can offer lower latency and power efficiency. These specialised coprocessors take shape in the form of application-specific integrated circuits (ASICs) and field-programmable gate arrays (FPGAs).

FPGA devices have a number of advantages over ASICs. Mainly, FPGA devices can be re-configured to implement any logic function, granted the device has enough resources available. Combined with FPGAs being freely available off-the-shelf, the faster time-to-market and better adaptability of FPGA devices, compared to their ASIC counterparts, are a great advantage to many systems [4]. These characteristics also pave the way for open-sourced hardware initiatives, allowing for accelerators to improve with community support and open up avenues for academic exploration.

One such accelerator is the NVIDIA Deep Learning Accelerator (NVDLA), a highly configurable industry-grade accelerator designed to speed up model inference [5]. The project includes the hardware description in register-transfer level (RTL) form written in Verilog and a software ecosystem containing a runtime environment and model compiler. While the project is no longer in active development [6], the combination of hardware scalability and dedicated software stack serves as a good model for end-to-end deployment and optimisation in industrial DL accelerators. However, despite the promise of NVDLA, there are various reasons why it might not fully realise its potential on FPGA, some of which include:

- **Design focus:** NVDLA’s design is based on an ASIC implementation, with the model integrated into NVIDIA’s own Jetson line of embedded computers [7]. Thus, due to the lack of direct optimisations, the documentation states that the performance may not be “directly comparable against other FPGA-based deep learning accelerators” [5].
- **FPGA clock speed:** FPGAs operate at clock speeds in the 100–200 MHz range. To compare, modern CPUs operate in the range of a few GHz.
- **Resources required:** As mentioned previously, an FPGA’s ability to represent logic is limited by the resources available on the device. Related work on NVDLA suggests that its FPGA implementation becomes quite costly in terms of resources as the configuration scales up [8], which means you would either need to buy a higher-range FPGA or deal with the limited throughput of the downscaled accelerator.
- **Bare-metal control:** Developers may want to initially bypass the overhead of using the software stack for simplified development, better control, and flexibility of the system [9]. This may lead to missing out on optimisations and difficulties with managing memory and handling the complex hardware configurations of the accelerator. NVIDIA has also not

provided adequate support for bare-metal implementations, with the official documentation lacking clarity in several areas.

1.2 Aims

This project aims to explore whether NVDLA can provide benefit to accelerating DL inference on the FPGA platform. This will be done by utilising the `nv_small` model of the architecture as a coprocessor, controlling the flow with direct bare-metal control. The performance gains will be evaluated by comparing the execution speed to that of a basic single-threaded CPU implementation. This project also aims to contribute towards the open-sourced ecosystem for NVDLA by providing the work done in this project on GitHub.

1.3 Objectives

To achieve the aim outlined above, we will lay out the following objectives:

- Create a valid, synthesizable design using the accelerator for the target FPGA.
- Implement a tensor library to be used in the software implementation and as input to the accelerator.
- Develop a basic single-threaded CPU convolution operation.
- Create functions to abstract away the register programming of the accelerator to perform a convolution operation.
- Compare the performance of the two implementations by measuring the time taken to perform convolution operations that may happen in a CNN model.

1.4 Success Criteria

To better gauge that an objective has been met, it is important to lay out the success criteria for some of them.

- Valid design - Success here will be measured by the completion of ‘sanity’ tests without error.
- Tensor library - A successful tensor library will allow us to specify any input tensor through ‘get’ and ‘set’ operations. It may also be required to transform the tensors into a valid format for the accelerator.

- NVDLA convolution - Success will be measured by comparing with the result of the CPU convolution.
- Compare performance - We will be comparing the performance of the convolution in terms of seconds to complete the operation. We will be comparing with a range of different feature and weight tensor sizes and then emulating the convolutions from a known CNN model.

1.5 Structure

The following chapters in this report will be organised in the following way. Chapter 2 will outline the technical background details about machine learning, deep learning, and hardware acceleration in deep learning. Chapter 3 will discuss the essential details of NVDLA in terms of its hardware architecture and software ecosystem. Chapter 4 will then display the implementation details used to generate results, both on the hardware level and on the software level. Chapter 5 will show and discuss the results generated from the previous chapter. Finally, chapter 6 will include a summary of the project as a whole, with discussion about future work and a personal reflection.

Chapter 2

Background

This chapter provides the technical background concepts involved with this paper. We will begin with an overview of deep learning, specifically convolutional neural networks. We will then discuss background concepts such as design considerations and strategies employed in deep learning accelerator architectures.

2.1 Convolutional Neural Networks

Convolutional neural networks (CNNs) are one of many architectures described as *deep learning* architectures. Deep learning (DL) is a subset of machine learning (ML), a broad topic that includes many techniques for machines to gain intelligence through the use of data. What distinguishes DL from ML is that due to the more complex characteristics of deep neural networks, they can automatically learn a representation of features from raw data without manual feature extraction [10]. This is a powerful development, as it allows for an end-to-end system without manual configuration of a feature space. This section will discuss the individual components of CNNs and how they are combined to create a model.

2.1.1 Artificial Neural Networks

Artificial neural networks (ANNs) form a basis for CNNs. ANNs are loosely modelled after the human brain, transmitting signals from one neuron to the next, with the strength of the signal based on its input and experience. The basic building block is the neuron model, which is illustrated in Figure 2.1. Generally, a neuron works by taking a weighted sum of the input vector, plus a bias term, and processing the result with a specified *activation function*. The weights in the weighted sum are typically what is modified during training and are what we refer to as "parameters" in a model.

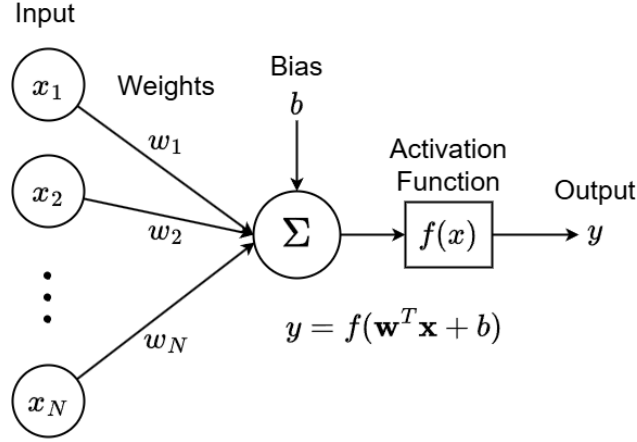


Figure 2.1: Artificial neuron model. The neuron takes the sum of $x_i \times w_i$ for each input i (which can be expressed as the scalar product of two vectors $\mathbf{w}$ and $\mathbf{x}$) plus a bias term, feeding the result to an activation function.

Activation Functions

There is a wide range of activation functions that are utilised for ANNs, the most basic of which is a simple linear activation, which is defined as:

$$f(x) = x$$

Introducing a non-linear activation function allows models the capability to learn more abstract patterns in data [11]. A recent survey by Kunc et al. [12] listed 400 of these activation functions, transforming the input given in a variety of different ways. Traditional activation functions include logistic sigmoid and tanh, however they have fallen out of fashion due to the high computational complexity and saturated outputs for high magnitude values of x [11]. Their mathematical expressions are:

- Logistic sigmoid:

$$f(x) = \frac{1}{1 + e^{-x}}$$

- Tanh:

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

The most commonly adopted activation function is ReLU, which stands for rectified linear unit. ReLU is defined by the following function:

$$f(x) = \begin{cases} x & x \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

ReLU is a very computationally inexpensive function to calculate and has been shown to improve the rate of training convergence in various models (for example, the popular AlexNet [13]). There are several activation functions that utilise ReLU as a base, aiming to change aspects such as the way negative values of x are treated [11]. Figures for logistic sigmoid, tanh, and ReLU activation are shown in Figure 2.2.

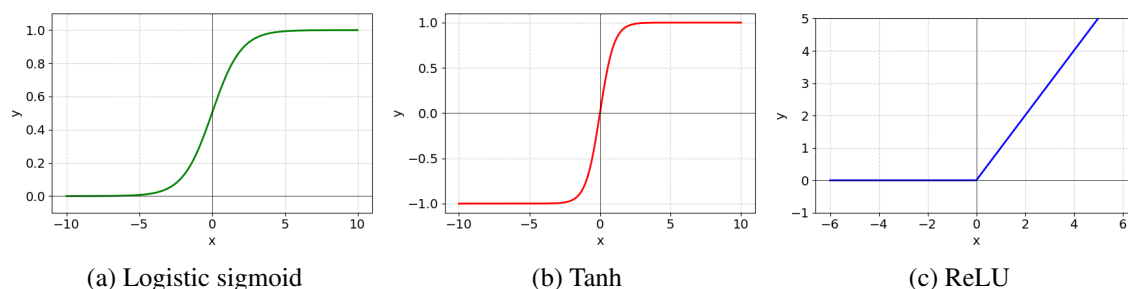


Figure 2.2: Graphs of activation functions.

Single-Layer Perceptron

A single-layer perceptron (SLP) is a type of model that contains one input layer and one output layer, as illustrated in Figure 2.3. A layer is built up of one or more neurons at the same 'depth' in the network, so SLPs can range from our single neuron example, to one with n outputs. SLPs can learn to distinguish linearly separable patterns, however they find difficulty as non-linearity and complex relationships are introduced.

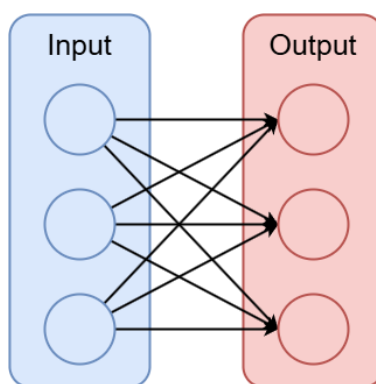


Figure 2.3: Single-Layer Perceptron.

Multi-Layer Perceptrons

Multi-layer perceptrons (MLPs)¹ are the next stage in complexity from SLPs. Due to the added hidden layers, as illustrated in Figure 2.4, they are able to model and learn complex patterns in

¹MLPs are commonly referred to as *fully-connected networks*

data. The compute power required for training and inference in MLP models scales rapidly as the number of hidden neurons increases. This is due to each neuron needing to propagate signals to every node in the next layer, and each of those connections having their own set of weights.

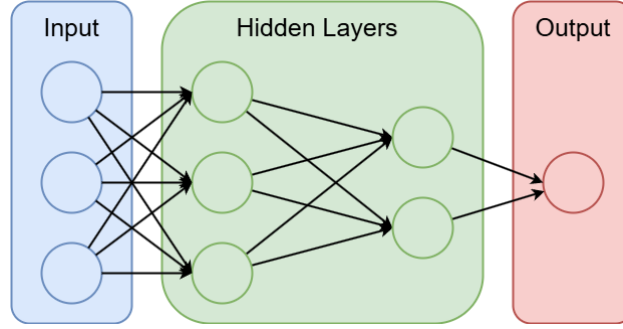


Figure 2.4: Multi-Layer Perceptron.

2.1.2 Convolution

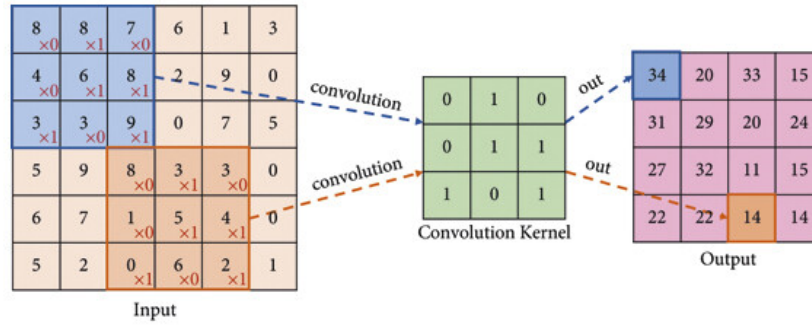


Figure 2.5: Visualisation of the convolution operation. The kernel slides spatially through the input image and sums the products of each corresponding input and weight element to produce the output. Source: [14].

A fundamental operation of CNNs is discrete convolution. Convolution is an operation heavily used in image processing, where we slide a grid of weights known as a *kernel* or *filter* across another grid's width and height, multiplying and accumulating the corresponding elements to produce a result as shown in Figure 2.5. The convolution operation in our case can be expressed as:

$$I'(m,n) = \sum_i \sum_j K[i,j] \cdot I[m-i,n-j]$$

where I' , I , and K represent the resulting image, original image, and kernel, respectively. The reason why the dot product is used is due to images containing a channel dimension, which we

take into account by ensuring weight kernels match the channel dimension size, striding only along the image's width and height. The values in the kernels are considered learnable parameters in a CNN, somewhat analogous to the single neuron mentioned previously. Convolution has a number of different hyperparameters, which give CNNs the capability to tune the output size of a convolution layer [15], this includes:

- **Kernel size:** In modern CNNs, each layer may have a different kernel size. In a study by Ahmed et al. [16], larger filter sizes are well-suited for earlier layers, decreasing the size as data moves closer to the output layer.
- **Stride:** Changing the stride will influence how the kernel moves from one part to the next. Larger strides will reduce the resolution of the resulting image.
- **Padding:** Padding is where we pad out the outer edges of our input image, increasing the perceived size of the input. There are many different types of padding, with the most common being zero-padding. Due to this, more padding will increase the resolution of the resulting image.

Using these parameters, we can formulise the resulting size of the output image:

$$size' = \frac{size + 2 \times padding - kernel_size}{stride} + 1$$

Much like a neuron, an activation function is typically used after the convolution function is applied. Every element in the output is passed through an activation function before reaching the next layer. It should be noted as well that if the kernel size matches the size of the input, a convolution behaves like a fully-connected layer.

2.1.3 Pooling

A method of reducing the complexity of a model is pooling. Pooling aims to reduce the dimensionality of images through sampling. Much like convolution, a filter is scanned across the image's dimensions. However, unlike convolution, the pooling filter does not have weights. Instead, pooling determines the output based on analysing its input, with common types of pooling being:

- **Min-pooling:** The minimum value in the kernel is chosen.
- **Max-pooling:** The maximum value in the kernel is chosen.
- **Mean-pooling:** An average is taken from the elements in the kernel.

The hyperparameters discussed in the convolution part (kernel size, stride, and padding) also apply to pooling due to their similar operations. An illustration of max-pooling can be seen in Figure 2.6.

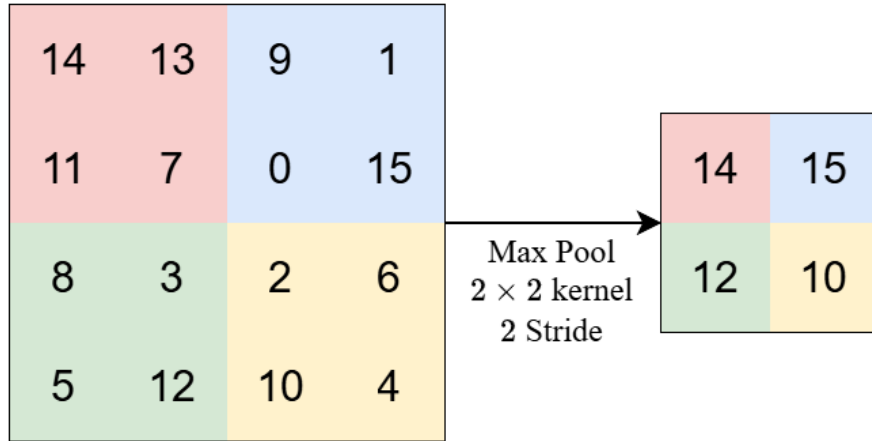


Figure 2.6: Visualisation of a max pooling operation, using a 2×2 kernel size and stride of 2.

2.1.4 Architecture

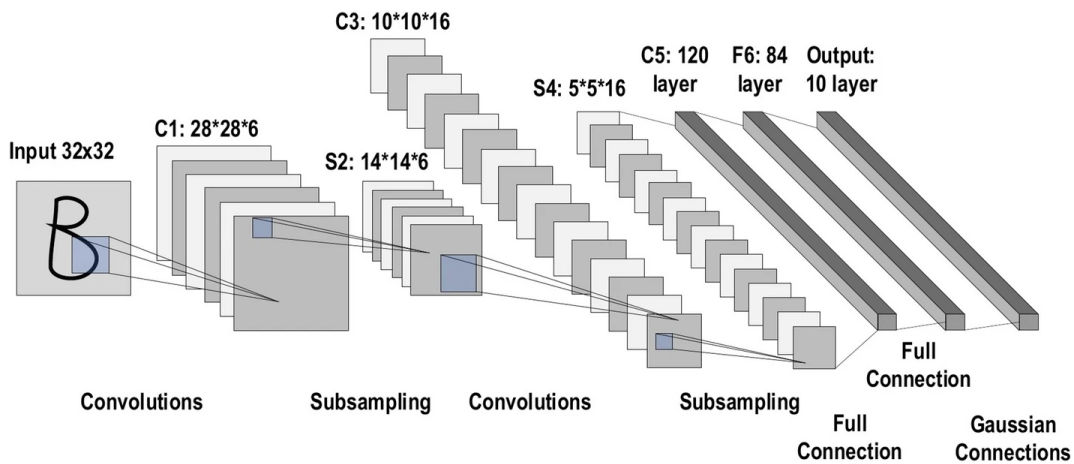


Figure 2.7: The architecture of the LeNet-5 CNN, as described in LeCun et al. [17]. Source: [18].

Typical CNN architectures employ a mix of the previously mentioned operations. To illustrate this, LeNet-5, as depicted in Figure 2.7, will be used as an example. LeNet-5 starts with six 5×5 kernels in the input layer, with the output of the convolution passing through a sigmoid activation function to reach C1. The image in C1 gets mean-pooled with a 2×2 kernel with a stride of 2. LeNet-5 then repeats this process on the subsampled image, performing another convolution layer (with sixteen 5×5 kernels) and mean-pooling (with a 2×2 kernel and stride of 2). The resulting image is flattened into a single vector and passed through an MLP with two hidden layers before reaching the output layer.

2.2 Field-Programmable Gate Array

Field-programmable gate array (FPGA) devices (illustrated in Figure 2.8) enable developers to reprogram the logical circuit to better suit their requirements. The architecture of FPGAs consists of an array of electrically programmable logic blocks that are interconnected together with a programmable fabric [19]. These logic blocks, also known as configurable logic blocks (CLBs), are generally comprised of look-up tables (LUTs) and edge-triggered flip-flops, however the exact structure of these logic blocks depends upon the device. LUTs are used to encode an n -input boolean logic function, and flip-flops are used as storage devices. FPGAs also commonly integrate dedicated *hardened* (non-programmable) blocks into the chip for common functions, including digital-signal processors (DSPs), memories, and CPUs.

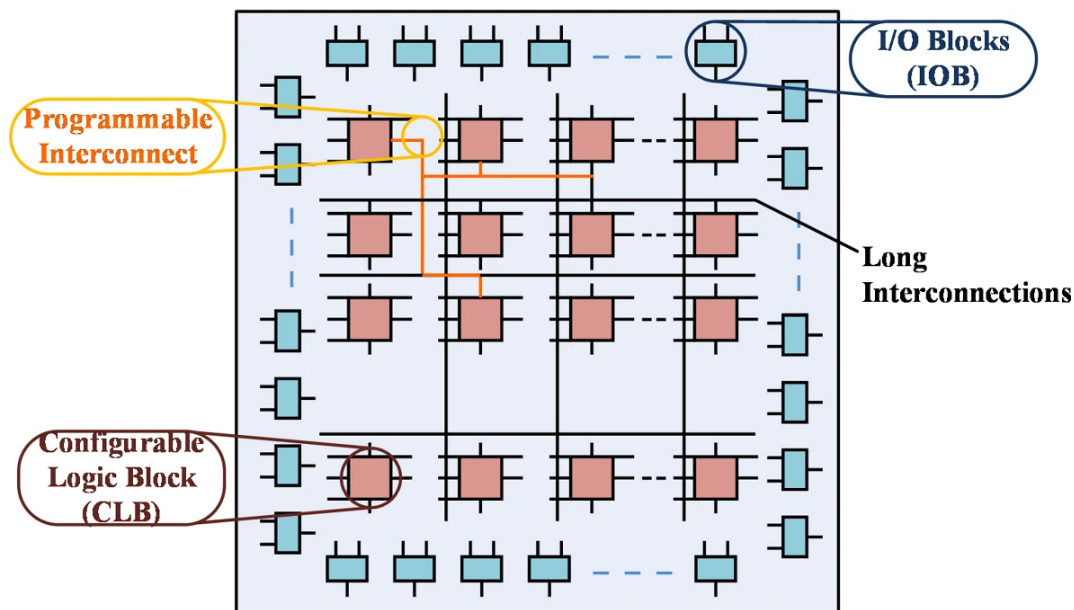


Figure 2.8: Basic FPGA architecture. Source: [20].

2.3 Deep Learning Acceleration

Utilising hardware accelerators to perform specific functions more efficiently is a widespread practice. In most high-performance systems, these accelerators are used as coprocessors [21] to the host CPU. This section will discuss the design objectives of deep learning accelerators (DLAs) and the common strategies they employ.

2.3.1 Design Objectives

Designing neural network hardware accelerators involves a number of design metrics and objectives to consider. In a book by Sze et al. [22], these are identified as:

- **Accuracy:** Accuracy is the quality of the result depending on what the DL model sets out to do. Using image classification as an example, the accuracy is the proportion of correctly classified images. Accuracy can be affected by the design of the accelerator, which may trade off accuracy in favour of better performance.
- **Throughput and latency:** Throughput is the amount of data that can be processed in a given time, whereas latency is the time it takes for a result to be generated. The overall throughput of a system can be increased by increasing the number of processing elements, increasing parallelism at a higher cost.
- **Energy efficiency and power consumption:** Energy efficiency can be defined as the amount of data that can be processed per unit of energy used, which is a concern for battery-powered systems and environmental impact. Power consumption is a metric relating to the total power used, which might concern cooling due to the thermal dissipation of the system.
- **Hardware cost:** The hardware cost relates to the financial cost of the architecture and the performance gain relative to the device cost. In terms of FPGAs, the cost of the device influences the availability of resources such as CLBs, DSP blocks, and on-chip memory, which may be required for the synthesised design.
- **Flexibility:** Flexibility relates to the DLA's ability to accommodate different parameters, such as neural network architecture, precision requirements, activation function, and other configurable aspects.
- **Scalability:** The metric of scalability assesses how well a design can utilise the increased amount of resources to achieve higher performance. Scalable designs should ideally enable the accelerator to be deployed across a range of devices, from low-cost embedded devices to cloud systems, by adapting to the resource constraints.

2.3.2 Design Strategies

The various design objectives for DLAs have led to the development of different design strategies for both accelerators and neural network models themselves. These strategies are commonly employed in neural network acceleration to optimise performance, cost, and other metrics.

Temporal and Spatial Architectures

The most common operations in DL models are multiply-accumulate (MAC) operations, which are readily parallelizable due to the commutativity of accumulation [23]. The main architectures for achieving high parallelism can be categorised as either *spatial* or *temporal*. As illustrated in Figure 2.9, both architectures make use of a large amount of processing elements (PEs) to work in parallel, composed of arithmetic logic units (ALUs) to perform these MAC operations, but differ in the origin of control and storage. Temporal architectures typically employ central control, whereas spatial architectures distribute the control and storage to each PE, allowing data to be propagated

between them [3]. The temporal architecture is seen in SIMD (single-instruction multiple-data) architectures such as GPUs and CPUs. Spatial architectures, on the other hand, have become increasingly adopted for specialised hardware.

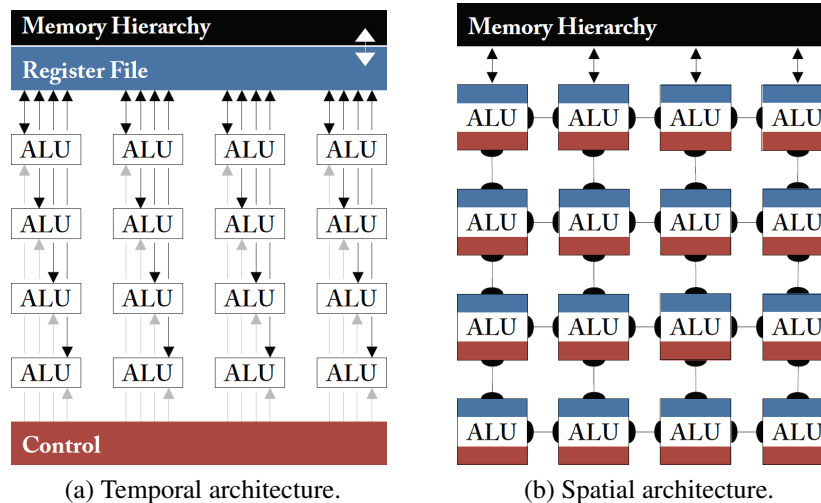


Figure 2.9: Visualisation of temporal and spatial architectures. Both consist of multiple arithmetic logic units (ALUs) working in parallel. However, spatial architectures allow data movement between ALUs, with each ALU having its own control logic and local memory, rather than relying on central control. Register file is used here to denote memory used in operations. Source: [23].

Precision Reduction

One significant area of research involves reducing the precision of data used in a model. Typical models are trained with 32-bit precision floating-point numbers [24] (denoted as FP32). However, through quantisation techniques, these numbers can be reduced to 8-bit integers (denoted as INT8) without significant loss in inference accuracy [25]. As illustrated in Figure 2.10, this leads to a significantly smaller energy and area footprint due to the relatively simple multiplication and addition hardware of low-precision integer values (compared to the higher precision floating-point values) and decreased volume of memory transfers [24]. The extreme case of quantisation is binarization, reducing the precision to a single bit, enabling MAC operations to be replaced with bitwise operations [26][27].

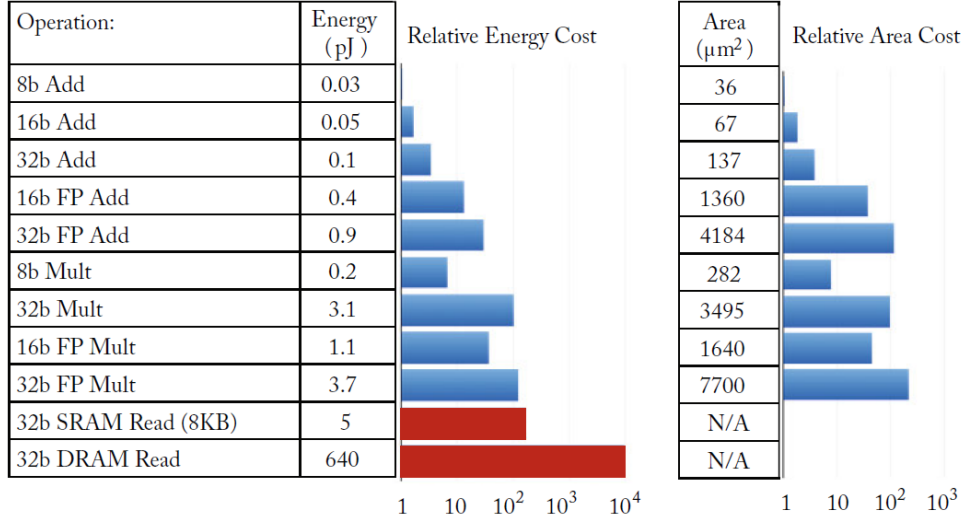


Figure 2.10: The relative energy and area cost for memory access, multiplication, and addition operations across different precision between integer and floating-point numbers. This figure is based on a 45 nm process. Source: [24].

Dataflow

Dataflow refers to the patterns of data movement in spatial architectures. By analysing the flow of data through the system, accelerators can attempt to maximise data reuse, improving efficiency by reducing off-chip memory access [28]. The data picked to be reused is said to remain *stationary* in the PE's local memory. Sze et al. [29] [30] presents the following taxonomy to categorise these different strategies:

- **Weight stationary:** This exploits weight-reuse by storing weights in the PE and propagating the accumulating partial sums through the array of PEs, as shown in Figure 2.12a. This advantages convolutions, as the filters are reused for every output. Batched fully-connected layers also benefit from this, as weights will be reused across batches. Examples of weight stationary architectures include Google TPUs [31] and NVDLA [5].
- **Output stationary:** The accumulation for the output is stored in the PE. This minimises the propagation of partial sums through the array of PEs. Examples of output stationary architectures include the DianNao family of ASIC accelerators [32].
- **Input stationary:** Similar to weight stationary, the input is stored in the PE, and the accumulating partial sums are propagated through the array of PEs. An example of input stationary includes the SCNN accelerator [33].
- **Row stationary:** To maximise reuse of all the above at once, the PEs perform 1D convolutions for a particular row of the input and kernel. As illustrated in Figure 2.11, weights are stored and reused in the PE, inputs are reused diagonally, and outputs are stored and

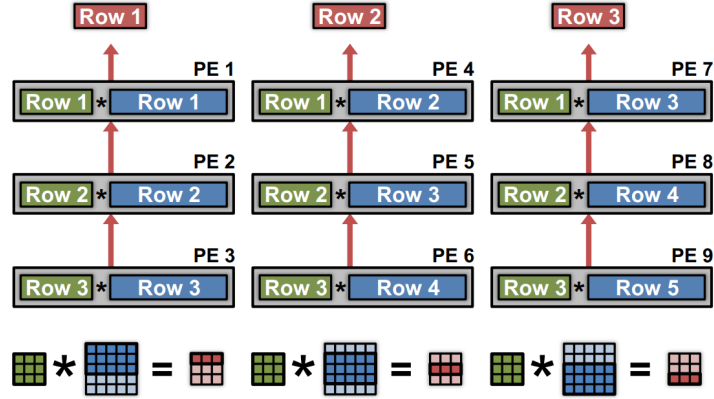
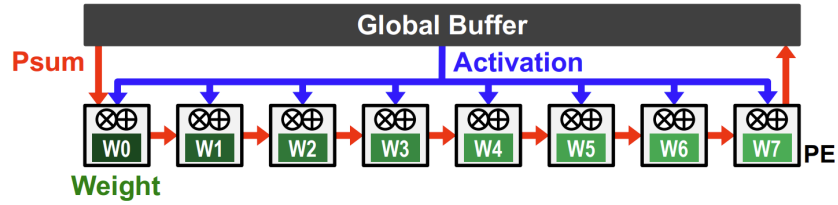


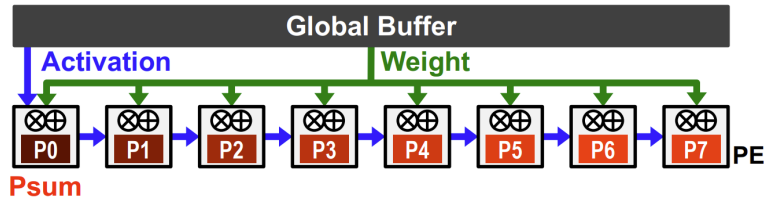
Figure 2.11: Visualisation of row stationary dataflow. Partial sums (red) of whole rows are calculated and propagated upwards, weights (green) are reused horizontally, and inputs (blue) are reused diagonally. Source: [36].

propagated upwards to calculate the full output row. Row stationary was introduced for the Eyeriss accelerator [34].

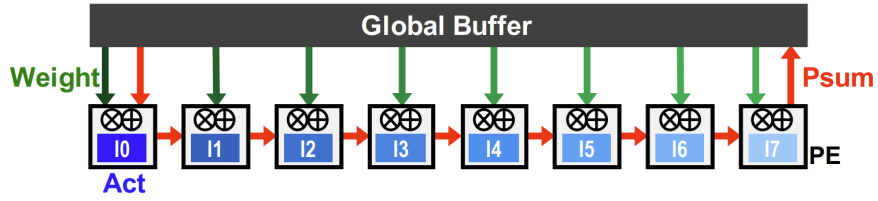
The optimal dataflow depends highly on the layer being accelerated. While typical hardware accelerators have a fixed dataflow, some, such as FlexNN, offer flexibility through software configuration at the cost of increased area overhead [35].



(a) Weight stationary.



(b) Output stationary.



(c) Input stationary.

Figure 2.12: Visualisations of the (a) weight, (b) output, and (c) input stationary dataflows for spatial architectures. *Activation* refers to the input data, *Psum* refers to the output partial sum, and *Weight* is the kernel weight. Colour gradients have been used to note different values of the same data type. Source: [36].

Chapter 3

NVIDIA Deep Learning Accelerator

The NVIDIA Deep Learning Accelerator (NVDLA) is an open-sourced hardware accelerator designed to efficiently accelerate DL inference by providing dedicated fixed-function hardware blocks and a full software ecosystem. The architecture (which employs a weight-stationary dataflow) is readily scalable and has a proven track record of improving performance as an ASIC, having been integrated into industry-grade products such as the NVIDIA Jetson Xavier and Orin platforms [7]. This section will provide an overview of the NVDLA hardware architecture and software ecosystem, based on information from the official documentation [5].

3.1 Repository Overview

The NVDLA source code can be found at GitHub [37], with the project being split into three repositories:

- **hw:** Hardware repository. Contains the hardware RTL description of the accelerator written in Verilog, along with a simulation model written in SystemC. The repository contains three branches. The `nvdlav1` branch contains the first version of NVDLA, which is a non-configurable, full-precision version of the design (`nv_large`). The `nv_small` and `master` branches both contain scalable designs, with the former being a stable release with verifications for the small configurations (`nv_small` and `nv_small_256`) and the latter containing more configurations (`nv_medium_512` and `nv_medium_1024`), which may be unstable. The configurations can be generated from their respective `.spec` files.
- **sw:** Software repository. Contains the runtime environment and neural network model compiler.
- **vp:** Virtual platform repository. Contains a system for prototyping software based on the SystemC simulation.

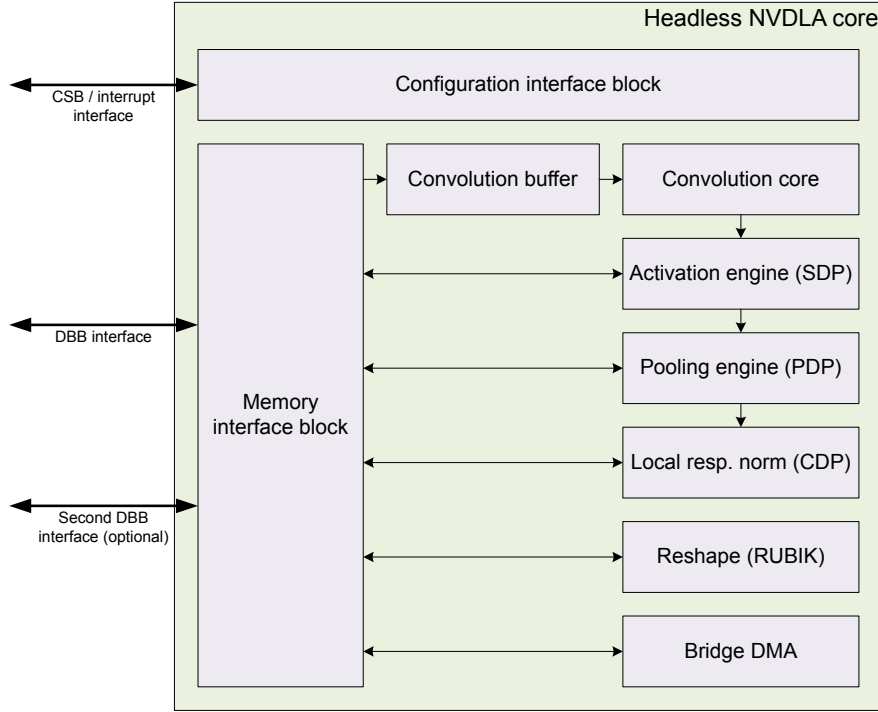


Figure 3.1: Figure of an NVDLA core. Source: [5].

3.2 Hardware

3.2.1 Core Overview

NVDLA offers the following fixed-function hardware blocks in its core architecture:

- **Convolution core:** As the name suggests, this is the primary engine that drives convolution operations between input feature data and a set of weights.
- **Single Data Processor (SDP):** This performs post-processing operations on individual data points. The SDP can be used to apply both linear and non-linear activation functions, batch normalisation, bias addition, and precision scaling. This is labelled as *Activation engine* in Figure 3.1.
- **Planar Data Processor (PDP):** The PDP is responsible for performing pooling operations, including max, min, and mean pooling. This is labelled as *Pooling engine* in Figure 3.1.
- **Cross-channel Data Processor (CDP):** This applies the local response normalisation function, a type of normalisation across the channel dimension rather than spatial dimensions. This is labelled as *Local resp. norm* in Figure 3.1.

- **Data Reshape Engine (RUBIK):** This engine performs data format transformations on a given tensor, such as slicing, concatenation, transpose, or reshaping.
- **Bridge DMA¹ (BDMA):** If the integrator wants to connect additional high-bandwidth SRAM² memory to NVDLA for performance, this engine is provided to enable copying data between system DRAM and SRAM.

An overview of the architecture for an NVDLA core is illustrated in Figure 3.1. This architecture supports two primary operating modes: *independent* and *fused*. Fused mode operates with the direct connections from the convolution core, SDP, PDP, and CDP, enabling these blocks to form a pipeline where data is transferred to the next layer via first-in first-out buffers (also known as FIFOs) instead of a memory round trip. Independent mode, on the other hand, allows blocks to operate independently from each other, utilising connections to the memory interface for data access.

3.2.2 Convolution Pipeline

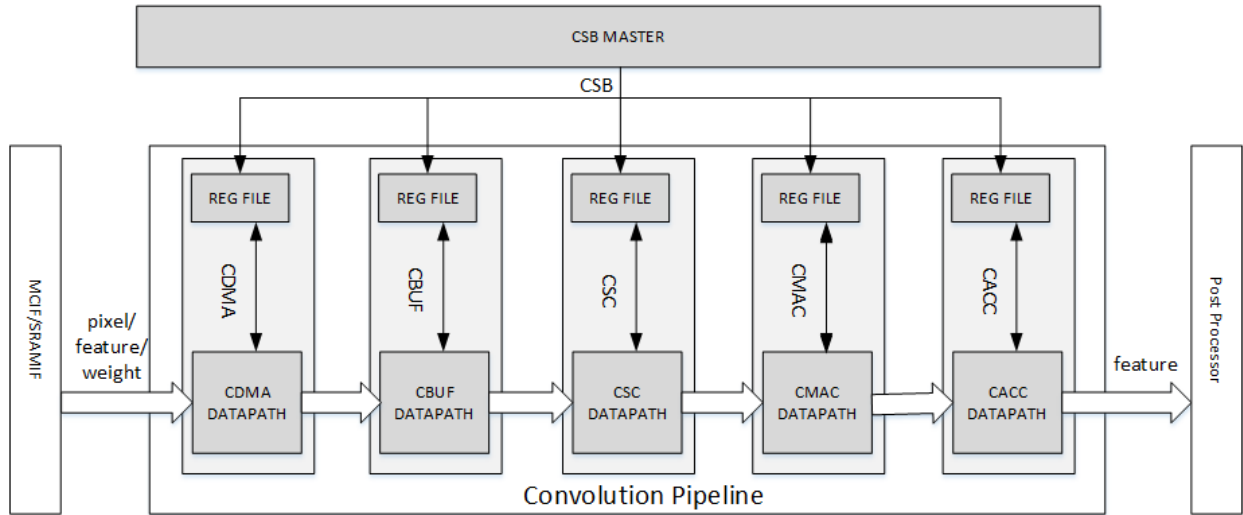


Figure 3.2: Convolution pipeline. Source: [5].

The convolution pipeline is one of the main logic pipelines within NVDLA. As illustrated in Figure 3.2, this pipeline is comprised of five stages: CDMA (Convolution DMA), CBUF (Convolution Buffer), CSC (Convolution Sequence Controller), CMAC (Convolution MAC), and CACC (Convolution Accumulator), with the output of the CACC fed into the post-processor (SDP). Each stage

¹DMA stands for direct memory access, a feature in systems where data can be transferred from one location to another without involvement of the CPU.

²SRAM stands for Static RAM, a type of RAM that is faster, higher cost, and uses more power than DRAM (Dynamic RAM).

has a register file connected to the CSB for configuring operational parameters, instructing these blocks how to operate.

NVDLA can support three types of convolution modes, which are:

- **Winograd convolution:** Winograd convolution is a type of convolution algorithm that aims to optimise the number of multiplications required in the standard convolution operation. NVDLA supports Winograd for $3 \times 3 \times C$ sized kernels.
- **Image-in convolution:** Convolutions on image formats have a special mode in NVDLA to improve resource utilisation when handling images, as images typically have a small amount of channels (roughly three to four).
- **Direct convolution (DC):** Direct convolution mode is the standard convolution operation. In this mode, NVDLA also offers feature batching capabilities in a mode called ‘Batching Convolution’ mode and support for sparsely compressed tensors for reduced memory bandwidth requirements.

Convolution Operation

The convolution operation performed by NVDLA closely resembles the same operation mentioned earlier, with the difference being the support for asymmetric padding and non-uniform strides. Asymmetric padding is where there is a different amount of padding on different sides of the feature cube, meaning instead of one padding parameter, we split this into four: left padding (LP), right padding (RP), top padding (TP), and bottom padding (BP). Non-uniform strides, on the other hand, signify that the stride in the X dimension (SX) is different from the stride in the Y direction (SY).

Firstly, this changes the calculation for the output dimensions to:

$$Width' = \frac{LP + Width + RP - Kernel_Width}{SX} + 1$$

$$Height' = \frac{TP + Height + BP - Kernel_Height}{SY} + 1$$

Secondly, to perform the convolution operation:

$$y_{w, h, k} = \sum_{r=0}^{R-1} \sum_{s=0}^{S-1} \sum_{c=0}^{C-1} x_{(w*SX-LP+r), (h*SY-TP+s), c} * wt_{r, s, c, k}$$

Where wt is the weight tensor and x is the feature input.

Convolution DMA

The first part of the pipeline, the CDMA, is responsible for fetching data from memory for the required convolution operation. CDMA fetches the feature data and weight data on two different read channels, storing the tensors into the convolutional buffer (CBUF). The CDMA utilises

different engines to fetch data, corresponding to the three different convolution modes mentioned previously.

Convolution Buffer

The CBUF is responsible for caching the tensors that will be used in the convolution operation and is comprised of multiple banks of a certain size, with the exact parameters depending on the particular hardware configuration used. Illustrated in Figure 3.3 is the CBUF of the full configuration NVDLA, containing 16 32 KB banks with 256 entries each. Each buffer acts as a circular buffer, incrementally adding new entries and wrapping back to overwrite the first entry once the maximum is reached.

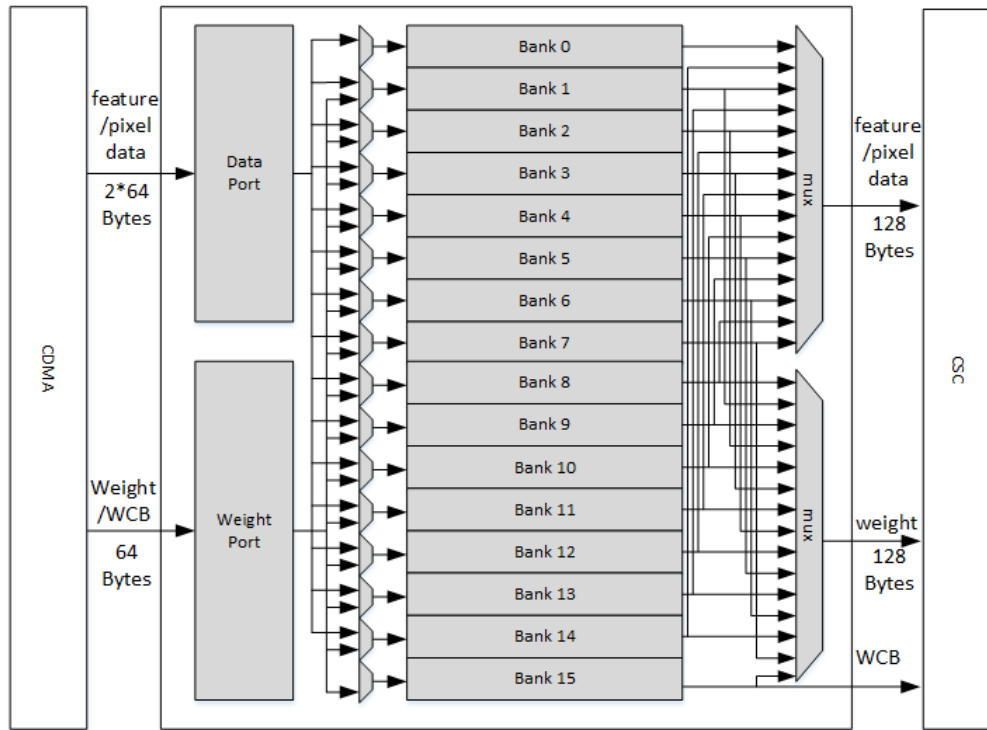


Figure 3.3: The convolution buffer used in the full configuration. Source: [5].

Convolution Sequence Controller

The CSC unit is used to load data from CBUF and pass it along as a packaged pair of feature and weight to the CMAC unit. This unit is responsible for controlling the convolution sequence and is comprised of three submodules:

- **Sequence generator:** This submodule generates the sequences, coordinating with the rest of the pipeline to decide when to read and send data packages.

- **Weight Loader:** This module loads weights from the CBUF, decompressing if needed (in the case that the weights are sparsely compressed), before sending them to the CMAC.
- **Data loader:** This module loads feature data from the CBUF and sends the data along to the CMAC.

Convolution MAC

CMAC performs the MAC operations for the convolution before passing the partial sum data along to the CACC. The CMAC is comprised of an array of MAC cells that each have a certain number of multipliers (and adders for the Winograd convolution). The number of MAC cells there are depends on configuration and is referred to as *Atomic-K*, while the number of multipliers in each cell is referred to as *Atomic-C*. Therefore, the total number of multipliers is $\text{Atomic-C} \times \text{Atomic-K}$.

Convolution Accumulator

The CACC is used to accumulate all of the partial sums generated from the CMAC unit, sending the results to the SDP. It consists of an adder array that accumulates the input into a higher precision result (for example, INT8 results are summed into an INT34 number), which is then clipped to the right precision and sent to the SDP for optional further processing and to perhaps write the result to memory (unless configured to pass the data along to the pooling engine).

3.2.3 Single Point Data Processor

The SDP is designed to perform post-processing operations on data. This unit can either be configured to receive data from the convolution core (specifically the CACC) or directly from memory via its own RDMA (Read DMA); this is also referred to as the on-flying and off-flying working mode, respectively.

Supported Operations

The operations supported by the SDP include:

- **ReLU activation:** The SDP can perform the ReLU activation, as described previously.
- **PReLU activation:** PReLU is similar to ReLU, with the behaviour of negative values of x changed to be a constant scale parameter. In mathematical terms this can be written as:

$$f(x) = \begin{cases} x & x \geq 0 \\ kx & \text{otherwise} \end{cases}$$

. The value of the scale factor k can either be a constant for every channel or unique for each.

- **Bias addition:** Bias addition is where a constant ‘bias’ value is added to an element. This bias parameter can either be a single bias for the whole tensor, a unique bias to be used for each channel, or a unique bias for every element.
- **Batch normalisation:** Batch normalisation is where each element is normalised around zero for stability reasons.
- **Element-wise operations:** The SDP is able to perform element-wise addition or multiplication between two tensors.
- **Non-linear activations:** Non-linear activation functions, such as logistic sigmoid and tanh, can be implemented via programming LUTs to mimic these functions.

Functional Blocks

The SDP utilises several blocks to perform the operations mentioned. Firstly, the M block is used to feed in the input tensors and can be configured to either read the data from the convolution core or memory via the M-RDMA.

Secondly are the BS/BN/EW (also known as X1/X2/Y) blocks. The BS and BN blocks have the same architecture and are used to perform ReLU, PReLU, bias addition, batch normalisation, and element-wise operations. The EW block, on the other hand, is used to perform all the operations that the BS and BN blocks do, as well as perform non-linear activation functions through LUTs. Each of these blocks has its own RDMA to read data from memory for operations where that might be required, such as element-wise tensor operations.

Lastly, the output data can either be transferred to the PDP or be written to memory through a WDMA (Write DMA).

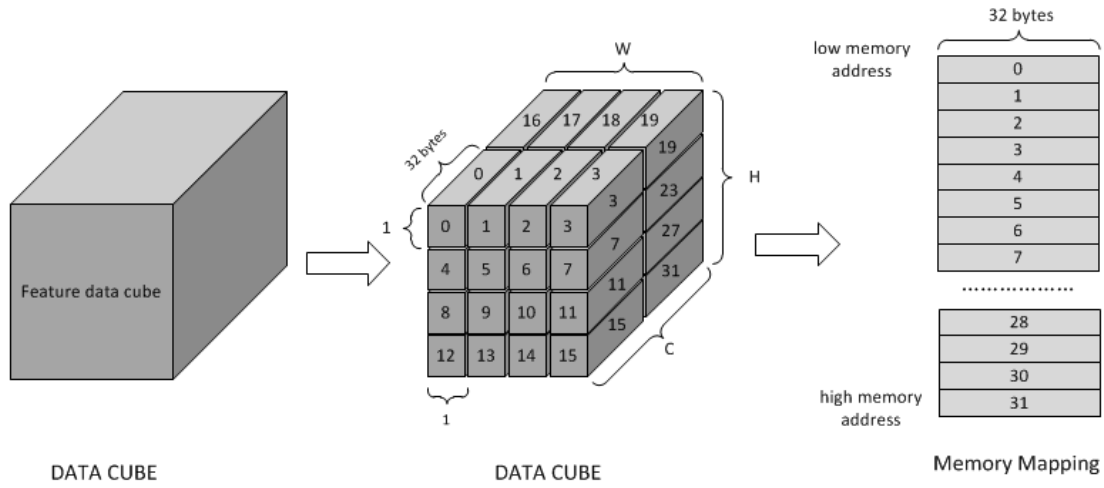
3.2.4 Feature Data Format

The feature data format (used for the input and output tensors) for NVDLA can be seen as a 3D data cube, with a width, height, and channel dimension. This cube is split into atomic cubes of size $1 \times 1 \times \text{Atomic-Cube-Size}$, as illustrated in Figure 3.4a, and scanned into memory in width, height, and channel order. This leads to the memory mapping following what is known as a pitch-linear format, where we end up with an array of atomic cube ‘lines’ forming a surface.

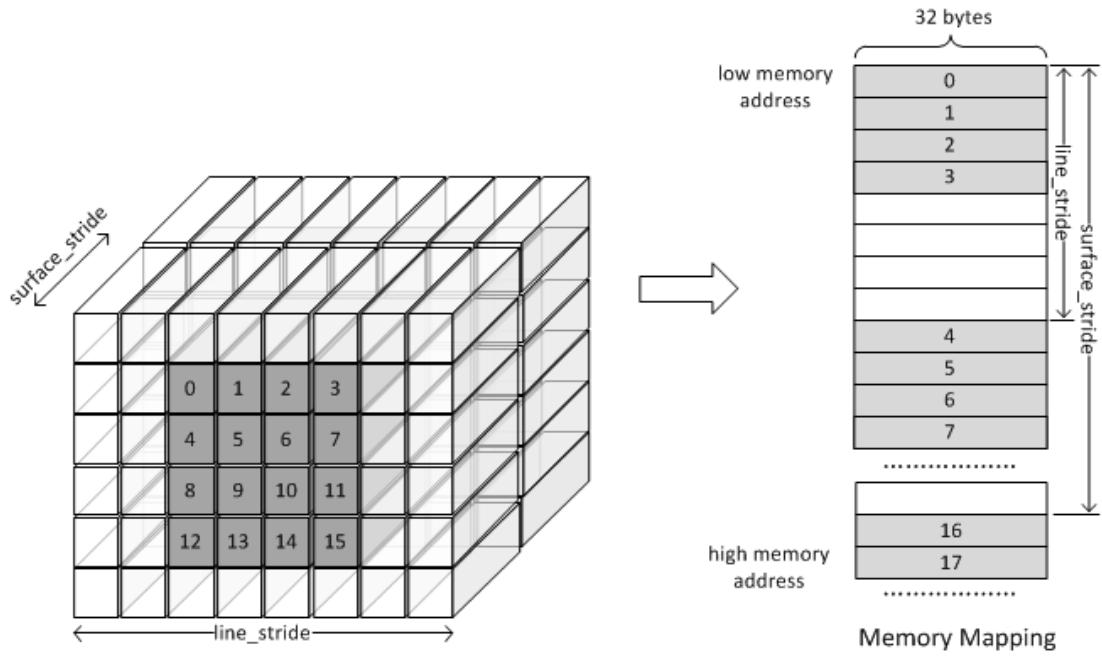
The feature format may not be compressed, or ‘packed’, as it is in Figure 3.4a. In cases illustrated in Figure 3.4b, the line stride and surface stride may need to be stated. It is important to note that for packed data, the line stride and surface stride can be calculated based on the following formulae:

$$\text{LineStride} = \text{Atomic-Cube-Size} \times \text{Width}$$

$$\text{SurfaceStride} = \text{Atomic-Cube-Size} \times \text{Width} \times \text{Height}$$



(a) Packed feature data



(b) Unpacked feature data

Figure 3.4: Feature data format and memory mapping, where Atomic-Cube-Size=32. Source: [5].

3.2.5 Integration

System Interfaces

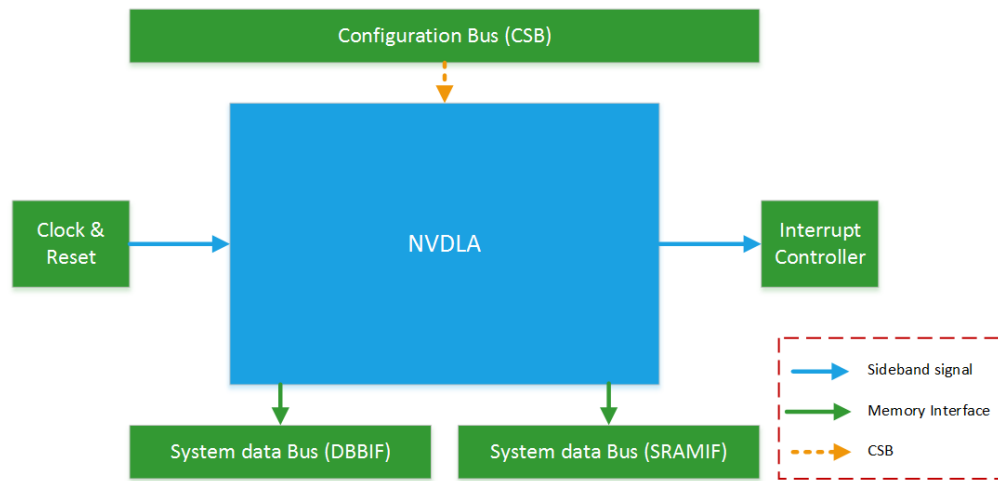


Figure 3.5: System diagram displaying NVDLA connections on a system on chip (SoC). Source: [5].

To integrate NVDLA onto a system-on-chip (SoC), as illustrated in Figure 3.5, the accelerator exposes the following interfaces:

- **Clock interfaces:** NVDLA has two independent clock domains: one for the core and the other for the configuration interface.
- **Reset interface:** This allows for a system reset to a known state.
- **System data interfaces:** The data backbone interface (DBBIF) and SRAM interface (SRAMIF) connect to the off-chip DRAM and on-chip SRAM, respectively. These interfaces are AXI4 (Advanced eXtensible Interface 4) compliant and are used to provide the data to be used in operations.
- **Configuration Interface:** This allows the system to program NVDLA's registers through the configuration space bus (CSB), configuring the operation parameters for each block.
- **Interrupt Interface:** The interrupt interface is used to notify the CPU when a task for a given block has completed.

Scalability Configuration Space

As mentioned previously, NVDLA is highly configurable to accommodate different area, cost, and flexibility needs. Each block of NVDLA is able to be configured independently, scaling them up

or down and enabling or disabling functions. The list of options can be found below:

- **Precision types supported:** Subset selection from binary, int4, int8, int16, int32, fp16, fp32, and fp64.
- **Convolution modes:** The Winograd and batching convolution mode features, as well as the sparse compression feature for input tensors, can all be independently enabled or disabled. Support for different image input formats can also be configured.
- **Non-linear Activation Functions:** The LUT for nonlinear activation function support in the SDP can be removed.
- **Second Memory Interface:** The second memory interface for SRAM support can be removed.
- **Engine Presence:** The PDP, CDP, RUBIK, and BDMA engines can all be independently removed.
- **Convolution Buffer Amount and Size:** Number of (from 2 to 32) and size of convolutional buffer data banks.
- **MAC Array Size:** Width (from 8 to 64, also known as Atomic-C) and height (from 4 to 64, also known as Atomic-K) of MAC array.
- **Activation Engine Size:** Number of activation outputs produced per cycle (from 1 to 16).
- **Pooling Engine Size:** Number of pooling engine outputs per cycle (from 1 to 4).
- **Local Response Normalization Engine Size:** Number of CDP outputs per cycle (from 1 to 4).

Small and Large Model

NVDLA integrations come in the categories of headless and headed, where headed integrations will utilise a microcontroller to interface with the accelerator rather than interfacing directly with the host CPU, delegating the management to the microcontroller to allow the CPU to perform other tasks. Illustrated in Figure 3.6 are two integrations of NVDLA, the “small” integration and “large” integration.

The small model is an example of a headless integration suitable for cost-sensitive devices attempting to reduce area and power consumption, categorised by not including added SRAM memory and by scaling down the accelerator instance itself.

The large model, on the other hand, is an example of headed integration with the inclusion of SRAM memory, which is more suitable for high-performance systems and offers a larger-scale accelerator to enable greater operational and parameter flexibility.

An overview of the differences can be seen in Table 3.1.

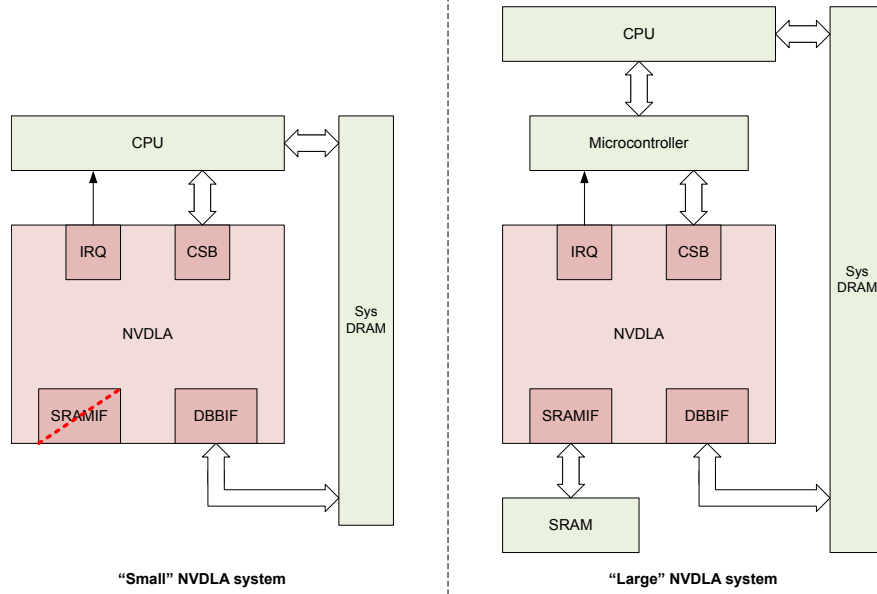


Figure 3.6: Comparison of NVDLA small and large systems. Source: [5].

Feature	NV_small	NV_large
Precision types supported:	INT8	INT8, FP16
Convolution modes:	DC	DC, Winograd, Batching, Compression
Non-linear Activation Functions:	No	Yes
Second Memory Interface:	No	Yes
Engine Presence:	PDP, CDP	PDP, CDP, RUBIK, BDMA
Convolution Buffer Amount and Size:	32, 128 KiB	16, 512 KiB
MAC Array Size:	8×8	32×64
Activation Engine Size:	1	16
Pooling Engine Size:	1	8
Local Response Normalization Engine Size:	1	8

Table 3.1: Comparison of NVDLA configuration options for nv_small and nv_large. Source: [37].

3.3 Software ecosystem

A significant factor towards NVDLA's status as an industry-grade accelerator is the software ecosystem. The inclusion of the software streamlines the deployment process towards using the accelerator as an inference engine. This process can be seen in Figure 3.7.

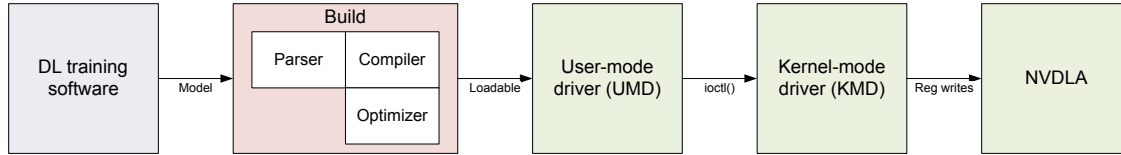


Figure 3.7: NVDLA software flow. Source: [5].

3.3.1 Compiler

The NVDLA compiler tool is responsible for creating an optimised sequence of hardware instructions corresponding to layers in a given pre-trained model³. The compiler will then allocate weights to memory, possibly quantising the model to a lower precision for efficiency. Further optimisations are made based on the configuration of the NVDLA instance, such as splitting convolution operations depending on available convolution buffer size. The output of the compiler is stored in a file format called NVDLA loadable.

3.3.2 Runtime Environment

The runtime environment is split between two drivers: the user mode driver (UMD) and the kernel mode driver (KMD). The UMD takes the NVDLA loadable file and submits it to the KMD. The KMD will then schedule the operations, handling the configurations for the registers in each functional block itself. These drivers can be integrated into any system with a portability layer, as illustrated in Figure 3.8.

³This model would have had to be pretrained with Caffe (Convolutional Architecture for Fast Feature Embedding), a deep learning framework [38] that is no longer supported as of 2018.

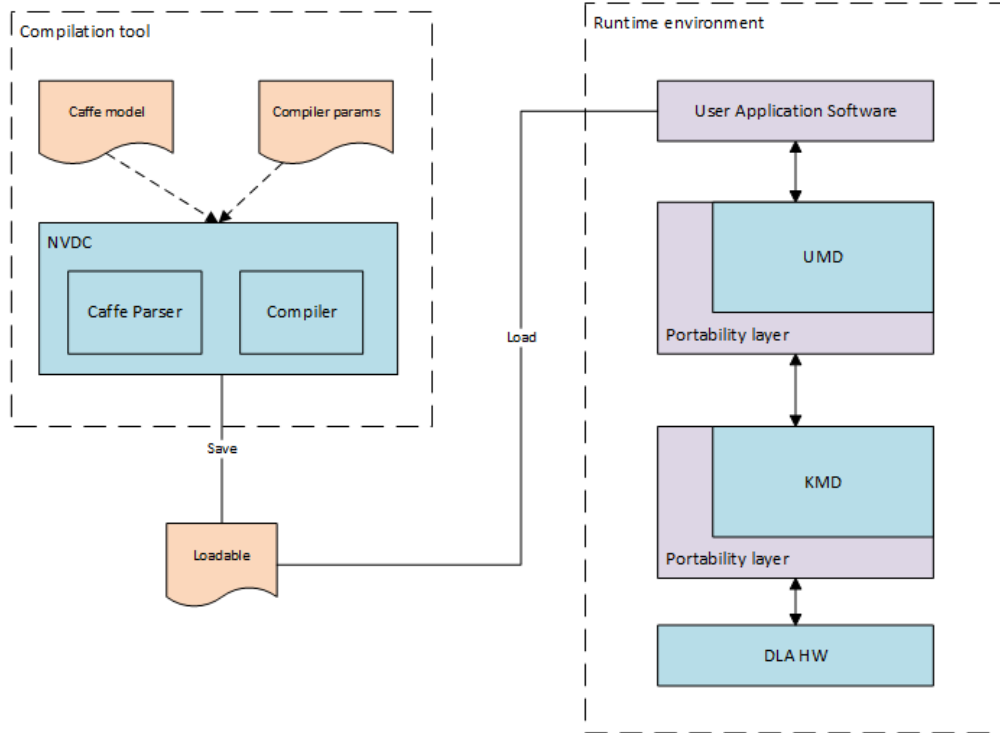


Figure 3.8: Figure showcasing an overview of the NVDLA software ecosystem. Source: [5].

3.4 Related Work

There has been a few instances of research involving porting the NVDLA design to the FPGA platform. One example is Lai and Huang [39] which utilises the `nv_small_256` accelerator on an FPGA to show that the performance of the YOLOv2-tiny model improves by around six times the frames per second (FPS) of an Intel Xeon CPU. Feng et al. [40] implement the LeNet-5 architecture on NVDLA bare-metal through a RISC-V core. They provide some details on certain register configurations, however they do not provide the source code for this work. Other types of work, such as Meena et al. [41], attempt to perform hardware analysis on NVDLA, benchmarking the performance difference through using a customised runtime and Winograd convolutions, both of which are shown to slightly increase the inference performance.

Chapter 4

Methodology

This chapter will outline the approach taken towards deploying the NVDLA architecture onto the FPGA platform and benchmarking the accelerator’s performance against a bare-metal software implementation.

4.1 Hardawre

4.1.1 FPGA Hardware Overview

The FPGA platform used for the purpose of this paper was the AMD Zynq UltraScale+ MPSoC (multiprocessor system-on-chip) FPGA, specifically the ZU9EG model. This FPGA has several resources available for creating programmable logic (PL), which will be utilised for the NVDLA implementation. Along with the PL, the FPGA also features an ARM Cortex-A53 APU (Application Processing Unit), which is part of what is known as the processing system (PS), which will be used to drive the accelerator, perform a software convolution, and extract timing data for both.

Programmable Logic

The exact number of resources available for the PL can be found in the datasheet for the FPGA [42], the most relevant of which include:

- 274080 CLB LUTs
- 548160 CLB flip-flops
- 2520 DSP blocks
- 912 Block RAM tiles

Evaluation Board

To enable utilisation of the FPGA, the ZCU102 evaluation board was used. This evaluation board comes with the FPGA device and allows developers to reprogram the fabric, debug programs, and view the output all via a USB connection. The board also comes equipped with 4GB of DDR4 SODIMM memory, which is used to hold the software and data, including the input, weight, and output tensors for the convolution.

4.1.2 Vivado Implementation Details

Vivado is a software environment designed by AMD for designing and synthesising hardware designs. For this project, the 2024.2 version of Vivado was used for integrating NVDLA with the PS and generating a loadable bitstream targeting the ZU9EG FPGA.

Git Integration

As Vivado does not have native version control support for project integration, the use of third-party scripts is required. Utilising scripts from the `vivado-git`[43] repository on GitHub enabled version control of the project through Git, which in turn allows the design to be hosted on GitHub. `vivado-git` saves the Vivado project details into a `.tcl` script, which can then be used to reconstruct the project in a fresh Vivado project. To ensure the script works, `vivado-git` demands that the Vivado project be hosted in a subdirectory of the main project folder, as illustrated in Listing 4.1:

```
1 PROJECT_NAME
2 |-- .git
3 |-- .gitignore
4 |-- project_name.tcl          # Project generator script
5 |-- src/                     # Tracked source files
6 |   |-- design/
7 |   |   |-- *.v
8 |   |   |-- *.vhd
9 |   |-- testbench/
10 |   |   |-- *.v
11 |   |   |-- *.vhd
12 |   |-- ...
13 |-- vivado_project/         # Untracked generated files
14 |   |-- project_name.xpr
15 |   |-- project_name.cache/
16 |   |-- project_name.hw/
17 |   |-- project_name.sim/
18 |   |-- project_name.srcs/
19 |   |-- sources_1/
20 |   |   |-- bd/             # BDs are regenerated from script
21 |   |   |   |-- my_bd/
22 |   |   |   |-- ...
23 |   |   |-- imports/hdl/    # BD wrappers are also regenerated
```



```

24 | | | | -- my_bd_wrapper.{v,vhd}
25 | | | | -- ...
26 | | | | -- ...
27 | | | | -- ...

```

Listing 4.1: Project hierarchy required for the vivado-git scripts. Source: [43].

NVDLA RTL

The reconfigurable accelerator can be found in the `master` branch of the NVDLA hw GitHub repository [37]. Utilising the `tree.make` setup detailed in the NVDLA documentation [5], the small NVDLA model can be generated using its predefined specifications (found in the `nv_small.spec` file).

To tailor the hardware description towards a synthesised FPGA implementation in particular, we must include a few global defines (macros) in a Verilog header. These macros (seen in Listing 4.2) ensure we are targeting an FPGA implementation rather than an ASIC implementation, which has different considerations in their design, such as clock gating.

```

1 `ifndef __COMMONS_VH__
2 `define __COMMONS_VH__
3
4 `define FIFOGEN_MASTER_CLK_GATING_DISABLED
5 `define SYNTHESIS
6 `define FPGA
7 `define NV_FPGA_FIFOGEN
8 `define VLIB_BYPASS_POWER_CG
9 `endif

```

Listing 4.2: Macros for the NVDLA FPGA implementation.

For the purpose of integrating NVDLA with the system, a premade hardware description wrapper was used, which can be seen in Appendix A.1. This wrapper encapsulated the NVDLA design with changes and definitions made for system integration in Vivado specifically:

- The outgoing interface (DBBIF) is defined as an AXI interface, making connections from this interface to other AXI ports compatible.
- An APB-CSB bridge is instantiated. This bridge is included in the RTL for NVDLA and allows the incoming interface (CSB) to be communicated with the APB protocol.

Block Design

Block designs on Vivado are used for system integrations. After importing the RTL files and NVDLA wrapper into Vivado, the system can be integrated with the PS in a block design.

1. Firstly, the PS (listed in Vivado as the Zynq UltraScale+ MPSoC IP) must be instantiated and configured to interface with the accelerator.

- Under PS-PL interfaces, we must expose one master and one slave AXI interface for data transfers between the PS and the PL. We will be using the high-performance (HP) interfaces on the full-power domain (FPD) to allow the accelerator to reach peak performance. The interfaces used are the *AXI HPM FPD* and *AXI HP FPD* for the master and slave, respectively.
 - We must also expose a PL to PS interrupt port, which will allow us to receive interrupt signals from NVDLA.
 - We can expose a clock output for the PS, which can be found under PL fabric clocks, to be used as the input clock for NVDLA. For the purpose of this project, this will run at a clock frequency of 100 MHz.
 - Under I/O configurations, the two Universal Asynchronous Receiver-Transmitter (UART) connections are required for debugging and monitoring over USB.
2. After instantiating the PS, we will connect the master port of the PS to the slave port of NVDLA. This will allow us to send register configuration data to the accelerator. As these ports utilise different protocols, an AXI-APB bridge IP after an AXI interconnect will need to be instantiated to facilitate communication. In the address editor tab, the base address to access this port is defined as A000_0000.
 3. The master port of NVDLA will connect to the slave port of the PS through an AXI Smart-Connect, which will allow the DMA engines of NVDLA to operate.
 4. The interrupt output for NVDLA will connect to the input interrupt port of the PS as mentioned previously.
 5. A processor system reset IP will be instantiated to facilitate a reset in NVDLA if signalled by the PS.

The finished block design can be seen in Figure 4.1. This block design was then turned into an HDL wrapper, which allowed it to be synthesised and exported as an XSA (Xilinx Support Archive) file.

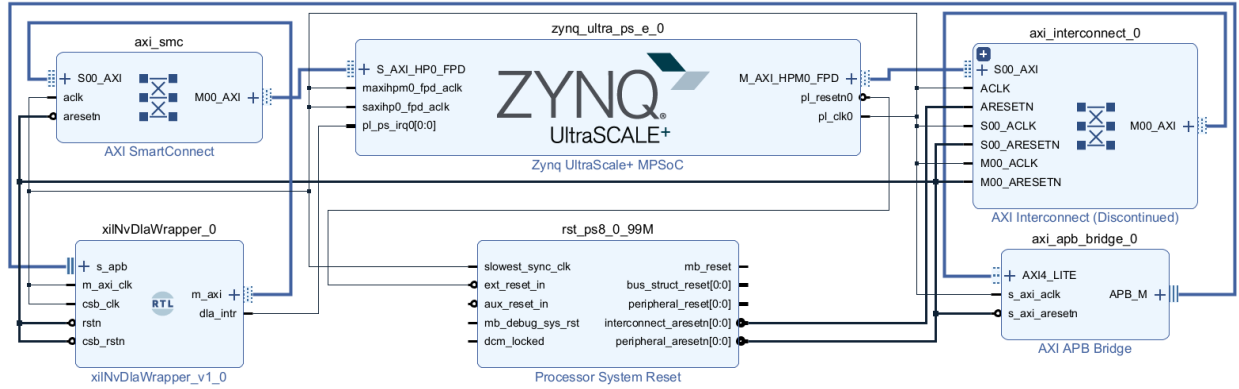


Figure 4.1: Block design for the FPGA-NVDLA integration.

For debug purposes, integrated logic analysers (ILAs) were utilised to monitor the signals coming in and out of the NVDLA wrapper as a waveform. This helped in debugging the integration of NVDLA beyond the output of software testing. Illustrated in Figure 4.2 is the block design with the added ILAs.

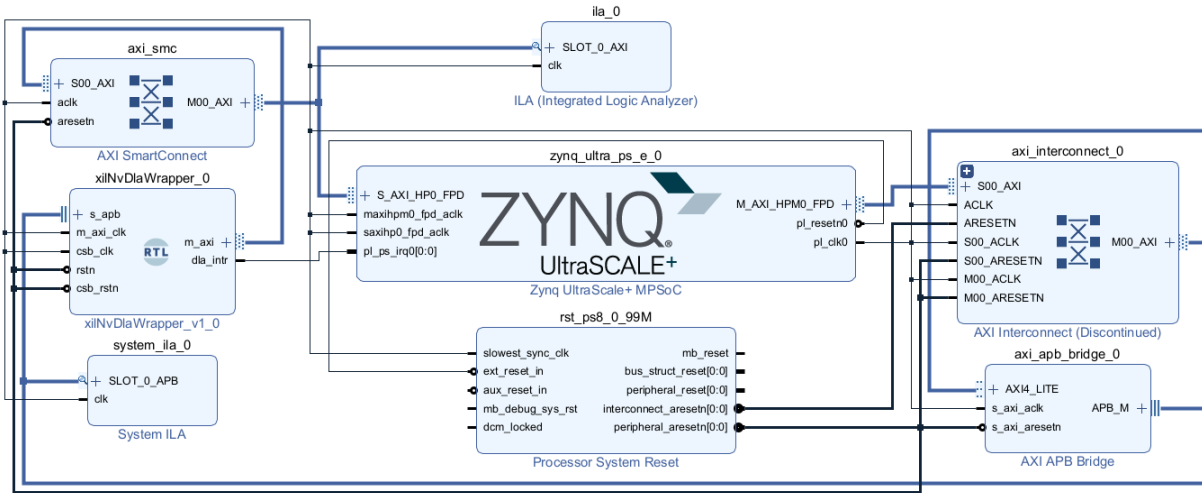


Figure 4.2: Block design for the FPGA-NVDLA integration with ILAs.

4.2 Software

4.2.1 Vitis Implementation Details

Vitis is an integrated development environment (IDE) designed to be the software complement of Vivado, used to program and debug software for FPGAs. For this project, the 2024.2 version of the Vitis Unified IDE was used for creating software to interface with the accelerator and extract timing information for testing.

Vitis Platform

The exported hardware design can be used to create a Vitis platform component, also known as a board support package (BSP). BSPs generate This platform will contain the hardware description, operating system (OS), and processor information. For this project, this is initialised as:

- **Operating system:** `standalone`. This ensures that the software is run bare-metal.
- **Processor:** `psu_cortexa53_0`. The software will be run on a single core (core 0) of the APU.

To extract timing information with the BSP, the `xiltimer` library must also be enabled. Now that the platform is initialised, an application component can be created with the platform to ensure the hardware design is loaded every time the application is run.

Vitis Application

The code for the software is encapsulated in a Vitis application component. Along with the source code, applications have launch configurations and a linker script. The launch configurations outline operations to be done before launching the application, which can be seen in Figure 4.3. The linker script has a “Stack Size” parameter, which is increased to `0x2000000` to accommodate large tensor dimensions.

```
Summary
Following operations will be performed before launching the debugger.

1. Resets entire system. Clears the FPGA fabric (PL).
2. Resets and clears APU reset.
3. Program FPGA fabric (PL) using the bit file (${workspaceFolder}\nvdla_tst_ila\ide\bitstream\NVDLA_FPGA_ILA.bit).
4. PS will be initialized using the FSBL file (${workspaceFolder}\nvdla_ila\export\nvdla_ila\sw\boot\fsbl.elf).
5. Tool will wait until (XFsbl_Exit) is hit before proceeding with rest of the operations.
6. The following processors will be reset and suspended.
   1. psu_cortexa53_0 (true)
7. All processors in the system will be suspended, and Applications will be downloaded to the following processors as specified above
   1. psu_cortexa53_0 (${workspaceFolder}\nvdla_tst_ila\build\nvdla_tst_ila.elf)
```

Figure 4.3: Vitis launch configuration summary.

4.2.2 Tensor Library

One of the main objectives for this project is to create a tensor library to perform software implementations of deep learning functions. This tensor library will provide a functional comparator for convolutions in NVDLA and provide an abstraction for the feature, weights, and output tensors. This section will outline the details of this tensor library.

Struct Definition

```
1 // Define the MAX SIZES for:
2 // W - Width
3 // H - Height
4 // C - Channel
5 // K - Batch
6 #define MAX_W 50
7 #define MAX_H 50
8 #define MAX_C 50
9 #define MAX_K 50
10 #define TENSOR_N MAX_W *MAX_H *MAX_C *MAX_K
11
12 typedef struct {
13     int batch_n;        // Batch size
14     int height_n;       // Height
15     int width_n;        // Width
16     int channel_n;      // Channel size
17     int8_t array[TENSOR_N];
18 } Tensor;
```

Listing 4.3: Tensor library struct definition.

As illustrated in Listing 4.3, the ‘Tensor’ type is comprised of size parameters for its dimensions (width, height, batch size, and channel size) and the main array. As the program is being run in an embedded context, the tensors are designed to be allocated on the stack rather than the heap.

Element Access

```
1 /*
2 * Set tensor value at specified index
3 */
4 void tensor_set(Tensor *t, int batch, int row, int col, int channel,
5               int8_t val) {
6
7     int idx = batch * (t->height_n * t->width_n * t->channel_n) +
8             row * (t->width_n * t->channel_n) + col * t->channel_n + channel;
9     t->array[idx] = val;
10 }
```

Listing 4.4: Tensor library setter definition.

To access specific elements of the tensor, for example, to set an element to a certain value as illustrated in Listing 4.4, the index must be calculated. The indexing used for the tensor library is a type of channel-last order, where dimensions are expressed as (batch, row, col, channel). The `tensor_get` utilises the same index calculation, with the inclusion of a boundary check so that out-of-bounds indices are treated as padding, as shown in Listing 4.5.

```

1      if (batch >= t->batch_n || row >= t->height_n || col >= t->width_n ||
2      channel >= t->channel_n
      || batch < 0 || row < 0 || col < 0 || channel < 0) return PADDING_VALUE;

```

Listing 4.5: Tensor library getter bounds-check.

Software Convolution

In order to compare the result of software convolutions with convolutions performed by the accelerator, the operation must be mathematically analogous to the convolution operation of the NVDLA.

```

1  /*
2  * Performs a convolution between feature and weight tensors
3  * storing results into dest
4  */
5  void tensor_convolution(Tensor *dest, const Tensor *feature, const Tensor *
   weight,
6                          int strideX, int strideY,
7                          int leftPad, int rightPad, int bottomPad, int topPad) {
8
9      dest->batch_n = 1;
10     dest->height_n = 1 + (topPad + feature->height_n + bottomPad - weight->
   height_n) / strideY;
11     dest->width_n = 1 + (leftPad + feature->width_n + rightPad - weight->
   width_n) / strideX;
12     dest->channel_n = weight->batch_n;
13
14     for (int ch = 0; ch < dest->channel_n; ch++) {
15         for (int r = 0; r < dest->height_n; r++) {
16             for (int c = 0; c < dest->width_n; c++) {
17                 int value = 0;
18
19                 for(int k_r = 0; k_r < weight->height_n; k_r++ ) {
20                     for(int k_c = 0; k_c < weight->width_n; k_c++ ) {
21                         for(int k_ch = 0; k_ch < weight->channel_n; k_ch++ ) {
22 int in_element = tensor_get(feature, 0, r * strideY - topPad + k_r, c *
   strideX - leftPad + k_c, k_ch);
23                 int w_element = tensor_get(weight, ch, k_r, k_c, k_ch);
24                 value += in_element * w_element;
25             }}}
26
27             if(value < -128) value = -128;
28             if(value > 127) value = 127;

```

```

29
30     tensor_set(dest, 0, r, c, ch, value);
31
32     }}}
33 }

```

Listing 4.6: Tensor library getter bounds-check.

Illustrated in Listing 4.6 is code used to perform a tensor convolution, saving the result to a destination tensor. Firstly, the function calculates the output dimensions, as shown in lines 9–12. A software emulation of the formula mentioned in Section 3.2.2 is performed to get the output value in lines 14–25. Finally, lines 27–30 will clip the output before saving the result into the output tensor.

Tensor Reshape

The output format of NVDLA is the previously described feature data format. In order to directly compare the output of NVDLA with the output of the direct convolution, utility functions were created to reshape the tensor between channel-last format and feature data format.

To transform channel-last to feature data format, the expected line stride, surface stride, and number of surfaces are calculated using the following formulae (note that 8 is used here as that is the size of the atomic cubes):

$$\begin{aligned}
 lineStride &= width \times 8 \\
 surfaceStride &= width \times height \times 8 \\
 surfaceCount &= \text{ceil} \left(\frac{channelSize}{8} \right)
 \end{aligned}$$

Then, the tensor is cloned, and each element from the cloned tensor is taken. Using the element’s channel index, the surface index and atom index can be found:

$$\begin{aligned}
 surface_idx &= \frac{c}{8} \\
 atom_idx &= c \mod 8
 \end{aligned}$$

The index in memory can then be calculated using the calculation in Listing 4.7, and the value can be set at that index.

```

1 int idx = k * surface_count * surface_stride +
2     surface_idx * surface_stride +
3     r * line_stride +
4     c * ATOMIC_C +
5     atom_idx;
6 t->array[idx] = value;

```

Listing 4.7: Index calculation for reshaping tensors.

To transform the format in the other direction, the same operation is done; however, the value is read from the index instead and set at the corresponding tensor index. The functions for transforming formats can be seen in Appendix A.2.

4.2.3 Software Macros

Software macros can be utilised to abstract away certain implementation-specific functions. This enhances readability and simplifies rewriting implementation-specific details, which is especially useful in an embedded context, where interactions with hardware may vary. Shown in Listing 4.8 are macros that expand to useful functions for reading, writing, and polling registers. It is worth noting that `xparameters.h` and `xil_cache.h` are libraries from the BSP, the former of which generates macros for memory addresses based on the exported hardware description, so `XPAR_XILNVDLAWRAPPER_0_BASEADDR` expands to `0xA000_0000` based on the base address of the APB port seen in the block design section (Section 4.1.2). Register offsets from the NVDLA base address were also turned into macros for readability and can be seen in Appendix A.3.

```

1 #include <xparameters.h>
2 #include <xil_cache.h>
3
4 #define NVDLA_BASE (void*) XPAR_XILNVDLAWRAPPER_0_BASEADDR
5
6 #define nvdlaRegWrite( __REG__ , __VAL__) *((volatile uint32_t*)(NVDLA_BASE +
   __REG__)) = __VAL__
7 #define nvdlaRegRead(__REG__) *((volatile uint32_t*)(NVDLA_BASE + __REG__))
8 #define nvdlaRegPoll(__REG__ , __VAL__) while(nvdlaRegRead(__REG__) != __VAL__
   )
9 #define flush_caches() Xil_DCacheFlush()

```

Listing 4.8: Macros for NVDLA functions.

4.2.4 Sanity Testing

To be certain that the NVDLA instance has been loaded into PL correctly, two ‘sanity’ tests were run. These tests help verify that the connections made in the hardware step work as expected.

CSB Sanity Test

The first test verifies the outgoing connection from the PS to NVDLA. Specifically, whether writing to a certain address corresponding to an NVDLA target register will result in that data being sent down the APB bus (and therefore the CSB). This test is essential in confirming that we can communicate with NVDLA from the PS to configure the hardware blocks and enable them. It should be noted that in order for direct communication with the NVDLA instance, the cache must be bypassed or flushed. Illustrated in Listing 4.9 is the sanity test and its use in the main function.

```

1

```



```

2 int sanity0()
3 {
4     print("Writing 0xFF to register...\n\r");
5     nvdlaRegWrite(NVDLA_CSC_D_ENTRY_PER_SLICE, 0xFF);
6
7     uint32_t val = 0;
8
9     print("-I: Reading register...\n\r");
10    val = nvdlaRegRead(NVDLA_CSC_D_ENTRY_PER_SLICE);
11
12    xil_printf("Value %d read from register.\n\r", val);
13
14    if(val != 0xFF) return -1;
15    return 0;
16 }
17
18 ...
19
20 int main()
21 {
22     ...
23     Xil_DCacheDisable(); // Disable cache while interfacing with the
24     accelerator
25     if(sanity0()) {
26         print("Error during test execution.\n");
27         return -1;
28     }
29     ...
30     return 0
31 }

```

Listing 4.9: Sanity test for PS-CSB connection.

The ILA core connected to this bus can also be utilised to verify that the data signal was propagated, as illustrated in Figure 4.4.

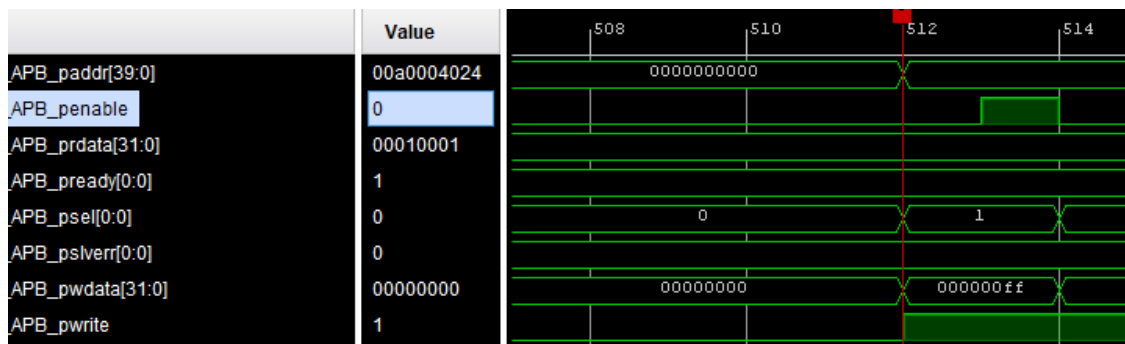


Figure 4.4: CSB sanity test output in ILA core, showing that the write signal was triggered on the APB bus to send the data of 0xFF to a specified address.

SDP Passthrough Test

As we have verified that the accelerator can be configured from the PS, a second sanity test will be performed to verify the outgoing connection from NVDLA to the PS. This connection is responsible for data access and allows the accelerator to read data from memory for processing. To test this, the SDP is configured to read data from one location and output the same data to another without any intermediate processing. This involves getting the SDP to read from memory (set to off-flying mode) pointing to an input tensor, bypass all of the processing modules (BS and BN - EW is not instantiated in `nv_small`), and write data to a destination tensor. The function relating to staging the passthrough test can be seen in Appendix A.4.

```
1 // Specify source_tensor, dst_tensor, RELU=0
2
3
4 NVDLA_Uutils_StagePassThroughSDP(&source_tensor, &dst_tensor, 0);
5
6 // Turn blocks on to complete operation
7 nvdlaregwrite(NVDLA_SDP_D_OP_ENABLE, 0x1);
8 nvdlaregwrite(NVDLA_SDP_RDMA_D_OP_ENABLE, 0x1);
9
10 sleep(5); // Await operation end
11
12 // Verify tensors are the same
```

Listing 4.10: Sanity test for NVDLA-PS DBBIF connection.

Listing 4.10 shows using the staging function and enabling the blocks to perform the operation. The ILA core again can be utilised to verify that data is being read and written from and to memory, as illustrated in Figure 4.5.

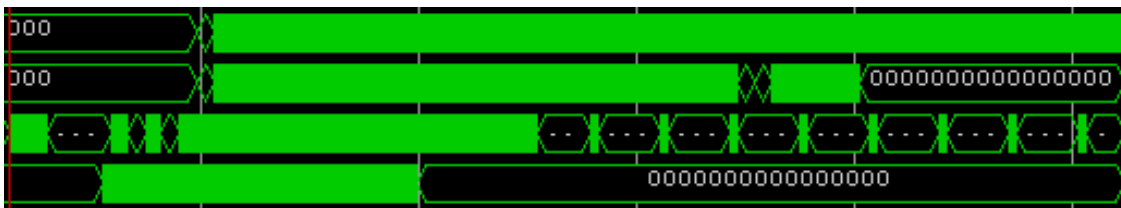


Figure 4.5: SDP sanity test output in ILA core, showing a large number of write (top two) and read (bottom two) related signals being triggered.

4.2.5 NVDLA Direct Convolution

Staging a direct convolution without the use of the software environment involves configuring every register in the convolution pipeline, as well as the hardware modules that come after (for example, the SDP may be configured to perform a ReLU activation). The entire staging function can be seen in Appendix A.5, this section will highlight the important details. The details on how to

program registers were inferred from a combination of the official documentation [5] (specifically from the hardware manual section), benchmark test traces from the GitHub repository [37] (which can be located in the `nv_small` branch under `/verif/tests/trace_tests/nv_small`), and the RTL source code.

Address Registers

The units in the convolution pipeline responsible for reading and writing data to and from memory require the addresses to be configured. These addresses are in 64-bit form, split into two 32-bit registers for the ‘high’ and ‘low’ halves. These addresses can be calculated using bitwise operations on the address, specifically the pointer for the array field in the tensor struct. Listing 4.11 shows an example of this with the feature input data for the CDMA.

```
1 // Address of feature data -> DAIN in NVDLA (Data IN)
2 uint64_t feature_addr = (uint64_t)feature->array;
3
4 nvdlaRegWrite(NVDLA_CDMA_D_DAIN_ADDR_LOW_0, feature_addr & 0xFFFFFFFF);
5 nvdlaRegWrite(NVDLA_CDMA_D_DAIN_ADDR_HIGH_0, (feature_addr >> 32) & 0
  xFFFFFFFF);
```

Listing 4.11: Address configuration for input data.

Decremental Parameters

Certain registers will require you to decrement the value you want to write. One example where this typically occurs is when specifying dimensions as illustrated in Listing 4.12.

```
1 nvdlaRegWrite(NVDLA_CSC_D_WEIGHT_SIZE_EXT_0, (weight->width_n-1) | ((
  weight->height_n-1) << 16));
2 nvdlaRegWrite(NVDLA_CSC_D_WEIGHT_SIZE_EXT_1, (weight->channel_n-1) | ((
  weight->batch_n-1) << 16));
```

Listing 4.12: Weight size register programming for the CSC.

Awaiting Direct Convolution

Once the direct convolution operation has been staged (all of the registers have been configured), every block involved in the pipeline needs to be enabled. Rather than being interrupt-driven, this implementation opts for the simpler approach of polling the SDP to see when the enable signal is low to signal when the operation has completed.

Testing

To verify direct convolutions, the output tensor is transformed and compared against the output of the software implementation for each run. To get a meaningful benchmark for the deep learning operations, we will compare against both fully-connected operations and convolution operations:

- **Fully-connected operations:** These operations are implemented as convolutions in NVDLA that occur between tensors that have the same width, height, and channel dimensions. This emulates fully connected network layers (such as MLPs), a common operation in deep learning.
- **Convolution operations:** By taking inspiration from the LeNet CNN model, the convolution layers from the model will be operated with.

The timing data will be extracted using the Xilinx timer interface, taking an average of three separate readings. The code used for testing can be found in Appendix A.6.

Chapter 5

Results and Evaluation

5.1 FPGA Implementation Statistics

5.1.1 Resource Utilisation Analysis

Resource	Available	Used	Percentage Used
CLB LUTs	274080	79022	28.83%
CLB Flip-Flops	548160	83260	15.19%
DSP Blocks	2520	32	1.27%
Block RAM Tiles	912	66	7.24%

Table 5.1: Post-Synthesis: Comparison of NVDLA resource utilisation against available resources on the ZU9EG FPGA.

Resource	Available	Used	Percentage Used
CLB	34260	16083	46.94%
CLB LUTs	274080	77333	28.22%
CLB Flip-Flops	548160	83139	15.17%
DSP Blocks	2520	32	1.27%
Block RAM Tiles	912	66	7.24%

Table 5.2: Post-Implementation: Comparison of NVDLA resource utilisation against available resources on the ZU9EG FPGA.

As previously mentioned, FPGAs have a limited amount of resources available for programmable logic. In this section, we will look at the resources that were utilised for the NVDLA small implementation in terms of CLB LUTs, flip-flops, DSP blocks, and BRAM tiles used. Using the reports

generated from Vivado when synthesising and implementing, the resource utilisation can be seen in Table 5.1 and Table 5.2 for the synthesis and implementation runs, respectively. It should be noted that these runs correspond to the hardware designs that do not utilise ILA cores for debugging. At first glance, the NVDLA small model seems to use a significant amount of resources, with 46.94% of CLBs utilised for this design. However, CLBs are comprised of both LUTs and flip-flops, so this number is broken down into 28.22% of total LUTs and 15.17% of total flip-flops being utilised, which indicates this FPGA can handle a higher-scaled instance of NVDLA or perhaps multiple instances of the `nv_small` model.

There is a clear underutilisation of DSP blocks (1.27%) and BRAM tiles (7.24%) in the design, which may be part of the reason why the documentation suggests NVDLA is not optimised for an FPGA implementation. By making better use of the resources available on the FPGA, the utilisation of LUTs and flip-flops can be reduced to improve the scaling capabilities of the accelerator.

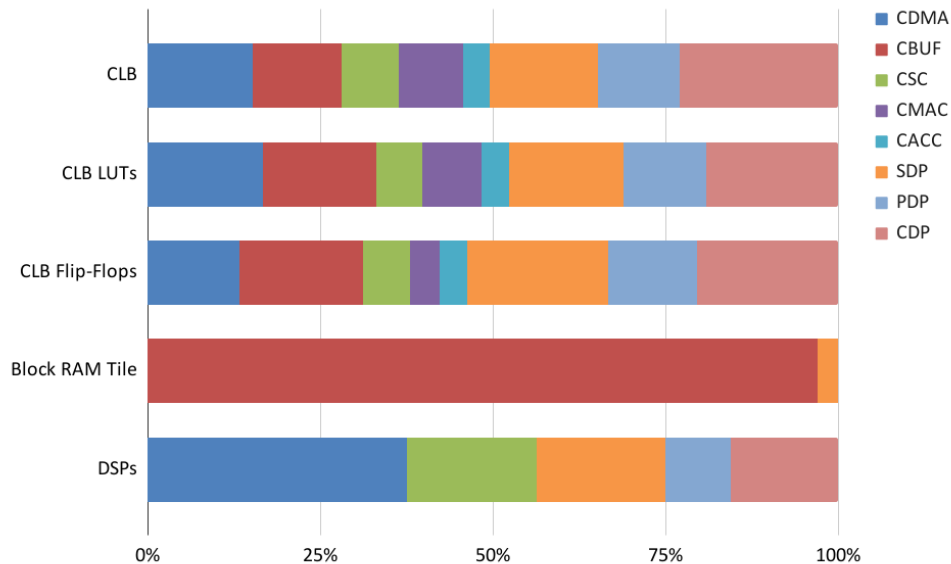


Figure 5.1: Chart showcasing the resource utilisation of the different hardware blocks.

Illustrated in Figure 5.1 is a chart showcasing the different proportions of resources used by the different blocks in NVDLA, based on Table 5.3.

A significant portion of resources is utilised by the CDP and PDP, which can both be removed to save the cost of instantiating them. Removing these engines would potentially free up 20000 LUTs and 12000 flip-flops, which could be utilised towards increasing the throughput of the main convolution pipeline.

The use of BRAM tiles is almost exclusively done by the CBUF block, creating two tiles for each CBUF bank. Despite this, the CBUF block still utilises a large amount of flip-flops and LUTs for its implementation. A small amount (two) of BRAM tiles are utilised by the SDP for FIFOs

Name	CLB	CLB LUTs	CLB Flip-Flops	Block RAM Tile	DSPs
CDMA	2249	11238	8773	0	12
CBUF	1892	11006	11801	64	0
CSC	1222	4507	4486	0	6
CMAC	1370	5726	2817	0	0
CACC	576	2691	2569	0	0
SDP	2315	11096	13553	2	6
PDP	1735	8087	8424	0	3
CDP	3400	12829	13503	0	5
Total	14759	67180	65930	66	32

Table 5.3: Resource utilisation by sub-module.

connected to RDMA.

DSP blocks are utilised most in the CDMA, which seems to primarily use these blocks to scale image input in a converter sub-unit. The CSC uses some for the data loader, the SDP for the PReLU activation, PDP for mean-pooling, and CDP for the local response normalisation function. The CMAC unit does not utilise DSP blocks, despite performing MAC operations.

It is clear that the resources utilised for the convolution pipeline are dominated by the CDMA, CBUF, and SDP. Upon close inspection of the breakdown, the SDP’s RDMA units contribute to over half of the CLBs utilised in the unit due to built-in multipliers and ALUs, with the numbers of resources totalling 6127 LUTs and 8541 flip-flops altogether. The CDMA uses around as many LUTs as the DC engine for the image-in convolution engine (about 2400 each), however the latter uses around twice as many flip-flops (1466 for DC and 2921 for image-in).

5.1.2 Timing Summary

Metric	Setup	Hold	Pulse Width
Worst Slack (WNS/WHs/WPWS)	1.510 ns	0.010 ns	3.500 ns

All user-specified timing constraints are met.

Table 5.4: Summary of timing analysis.

When implementing synchronous digital circuits, it is important to ensure there are no *timing violations*. Flip-flops in circuits require data to remain stable for a certain amount of time before and after triggering from clock signals to ensure that the data it captures and holds is stable. Data being processed in the logical circuits between flip-flops can take a variable amount of time depending on the path they take, meaning that data can take a variable amount of time to arrive from one flip-flop to the next. To ensure there are no violations, the worst-case slack is calculated. Slack

is a figure that can indicate how ‘comfortably’ a timing closure has been met, with negative values indicating a possible violation.

Worst negative slack (WNS) relates to setup time violations (the amount of time data is required to remain stable before the clock signal) and indicates the worst-case difference between the required setup time and the data arrival time. As illustrated in Table 5.4, the WNS is 1.510 ns, indicating that the timing constraint has been met with a comfortable margin. As setup time is most affected by the clock frequency, these results indicate that the design can possibly be re-implemented with a higher frequency than 100 MHz.

Worst hold slack (WHS) relates to hold time violations (the amount of time data is required to remain stable after the clock signal), which are a problem when data changes too quickly after a clock signal. The WHS has a positive value of 0.010 ns, indicating that the hold timing closure has been met.

Worst pulse width slack (WPWS) verifies that the clock signal meets the requirements for clock period, skew, and width of ‘high’ and ‘low’ pulses for the flip-flops. The WPWS also shows a positive value of 3.500 ns, indicating that there are no problems with the clock signal in this design.

5.2 Hardware against Software Convolution

5.2.1 Errors

Due to the difficulty of programming the individual registers and the poor documentation for NVDLA, there are various errors associated with the interface. Firstly, there is an error with convolution operations that require a vertical stride. In other words, the interface only supports horizontal convolution operations. This leads to poor accuracy and flexibility in possible operations, with the cause difficult to pinpoint. Therefore, testing is done with feature and weight data that have a height of 1, with the main comparison caused by altering width and channel size.

The second error with the interface is that every successful convolution operation requires a system reset before performing another one. The await convolution method gets stuck waiting for the SDP to become idle, which may indicate a stall in a preceding part of the pipeline. This freezing is also exhibited when input sizes get too large.

5.2.2 Fully Connected Layer Timings

Illustrated in Table 5.5 is the timing difference of separate fully-connected layer runs. There is a clear improvement in the time it takes to complete this layer, with the timing differences reaching consistently above 98%. During these runs, minimal variance could be observed.

Kernel dimensions	ARM elapsed (secs)	NVDLA elapsed (secs)	Timing Difference (Percentage)
8x64x1x8	0.00087046	0.00001184	98.64%
8x64x1x16	0.00172753	0.00002206	98.73%
8x32x1x8	0.00043672	0.00000780	98.21%
8x32x1x16	0.00086487	0.00001348	98.44%

Table 5.5: ARM vs. NVDLA elapsed time. Kernel dimensions are in the form Batches $\times$ Width $\times$ Height $\times$ Channels.

5.2.3 Convolution Timings

The following convolutions are based on layers of LeNet-5. LeNet-5 contains layers of 5×5 kernels, an attempt was made at simulating the convolutions at a single height level (to ensure the results can be verified) by setting the source width to 50 and the kernel width to 25.

Kernel Size	ARM elapsed (secs)	NVDLA elapsed (secs)	Timing Difference (Percentage)
16x16	0.03508164	0.00002700	99.92 %
8x8	0.00883928	0.00001030	99.99%
32x32	0.11312778	0.00008396	99.92%

Table 5.6: ARM vs. NVDLA elapsed time. Kernel dimensions are in the form of Batches $\times$ Channels.

As shown in Table 5.6, NVDLA vastly outperforms the single ARM-core in convolution operations, despite running at a much lower clocking frequency than it. Although these results might be slightly different when the height parameter is involved, it is certain that the higher throughput of the convolution pipeline leads it to outperforming the CPU implementation.

Chapter 6

Conclusion

6.1 Summary of Achievements

In this project, we were able to instantiate and deploy the `nv_small` specification of the NVIDIA Deep Learning Accelerator onto the widely used Zynq UltraScale+ MPSoC FPGA platform, hosting the design open-source on GitHub to improve the accessibility of future deployments on the same platform. A resource utilisation summary was extracted from this implementation, pointing towards optimisations that could be made for NVDLA on FPGA, along with investigations into where resources are utilised. The resource summary also showed that there is room on the particular FPGA model (ZU9EG) for an increase in the accelerator's scale. Along with the resource utilisation summary was the timing analysis, which suggests that there is room to increase performance further on this platform through an increase in clock frequency if required, with a positive slack for all timing closures for a 100 MHz clock rate.

In addition to the successful porting of the design, we were able to develop a bare-metal interface for deployment of convolution operations with NVDLA, bypassing the software ecosystem to gain more control over the accelerator. Through the tensor library written in this project, we are able to specify feature and weight tensors, pass them along to a direct convolution function to abstract away the register programming for the accelerator, and receive an output tensor generated from NVDLA.

Finally, through the Xilinx timer library, we were able to quantify the performance gained from utilising NVDLA bare-metal on the FPGA platform. Despite the errors in the interface, we were able to verify that the offloading procedure works and improves performance over the software implementation. The work done on this project will be made open-sourced to aid in future FPGA prototyping of the accelerator.

6.2 Critical Reflection

There are many areas that could have been improved upon during this project, mainly focusing on the software aspect. Due to the difficulty of understanding how to program registers, this has led to direct convolutions that require a height stride to not be usable. This is a massive flaw in the system, and I could not pinpoint the reason as to why it occurs in the time I had for this project. This led to the timing results being of much lower quality than that of related work and the reduced utility of the interface. To better understand how registers are programmed, it may have been better to try and generate some traces of simple convolutions with the software ecosystem myself.

Another area to improve upon would be establishing a proper structure for the interface. The current interface only implements SDP pass-through and DC operations. There is no real structure to these files, which may make it difficult for the software to be built upon in the future without mass refactoring. A lot of the software is also tightly coupled with Xilinx functions, such as print statements or Xilinx math functions (like `Xil_ceil()`), which may have been better structured as either software macros or only implemented given a certain macro.

6.3 Ideas for Future Work

One idea for future work would be a complete implementation of the software stack, allowing for more complex deep learning operations and layers that are able to take full advantage of the NVDLA architecture. The main goal would be to enable most DNN models to run on NVDLA for efficient deep learning inference. This would involve interfacing with the rest of the units.

Another direction for future work is the implementation of dynamically adaptive deep learning models, which are models that can adapt the computational structure of themselves based on the complexity of the given input. This work would require establishing compatibility and control for these models for both the hardware and the software, which an open-sourced accelerator like NVDLA lends itself to.

Bibliography

- [1] J. Holdsworth and M. Scapicchio, “What is deep learning?” 2024. [Online]. Available: <https://www.ibm.com/think/topics/deep-learning>
- [2] R. Rahman and D. Owen, “The training compute of notable ai models is doubling roughly every five months,” 2024, accessed: 2025-04-16. [Online]. Available: <https://epoch.ai/data-insights/compute-trend-post-2010>
- [3] P. Dhilleswararao, S. Boppu, M. S. Manikandan, and L. R. Cenkeramaddi, “Efficient hardware architectures for accelerating deep neural networks: Survey,” *IEEE Access*, vol. 10, pp. 131 788–131 828, 2022.
- [4] J. Xu, “Comparison and application of fpga and asic in digital system design,” in *Proceedings of the International Conference on Decision Science & Management*, ser. ICDSM '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 216–220. [Online]. Available: <https://doi.org/10.1145/3686081.3686117>
- [5] NVDLA Official Documentation. [Online]. Available: <https://nvdla.org/>
- [6] NVIDIA Community Support, Community post detailing NVDLA is no longer in active development. [Online]. Available: <https://forums.developer.nvidia.com/t/support-for-nvdla/261779/3>
- [7] NVIDIA Jetson specification. [Online]. Available: <https://developer.nvidia.com/embedded/jetson-modules>
- [8] A. Veronesi, D. Bertozzi, and M. Krstic, *Assessing the Configuration Space of the Open Source NVDLA Deep Learning Accelerator on a Mainstream MPSoC Platform*. Springer International Publishing, 2021, p. 87–112. [Online]. Available: http://dx.doi.org/10.1007/978-3-030-81641-4_5
- [9] R. Summers, M. Yeary, H. Sigmarsson, and R. Rincon, “Architecture study for a bare-metal direct conversion radar fpga testbench,” in *2021 IEEE Radar Conference (RadarConf21)*, 2021, pp. 1–5.
- [10] C. Janiesch, P. Zschech, and K. Heinrich, “Machine learning and deep learning,” *Electronic Markets*, vol. 31, no. 3, p. 685–695, Apr. 2021. [Online]. Available: <http://dx.doi.org/10.1007/s12525-021-00475-2>

- [11] S. R. Dubey, S. K. Singh, and B. B. Chaudhuri, "Activation functions in deep learning: A comprehensive survey and benchmark," *Neurocomputing*, vol. 503, pp. 92–108, 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0925231222008426>
- [12] V. Kunc and J. Kléma, "Three decades of activations: A comprehensive survey of 400 activation functions for neural networks," 2024. [Online]. Available: <https://arxiv.org/abs/2402.09092>
- [13] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems*, F. Pereira, C. Burges, L. Bottou, and K. Weinberger, Eds., vol. 25. Curran Associates, Inc., 2012. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf
- [14] Y. Dong, Y. Zhuang, and X. Yan, "Data-driven quality prediction of batch processes based on minimal-redundancy-maximal-relevance integrated convolutional neural network," *Mathematical Problems in Engineering*, vol. 2021, no. 1, p. 6842835, 2021. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1155/2021/6842835>
- [15] A. Younesi, M. Ansari, M. Fazli, A. Ejlali, M. Shafique, and J. Henkel, "A comprehensive survey of convolutions in deep learning: Applications, challenges, and future trends," 2024. [Online]. Available: <https://arxiv.org/abs/2402.15490>
- [16] W. S. Ahmed and A. a. A. Karim, "The impact of filter size and number of filters on classification accuracy in cnn," in *2020 International Conference on Computer Science and Software Engineering (CSASE)*, 2020, pp. 88–93.
- [17] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [18] L. Alzubaidi, J. Zhang, A. J. Humaidi, A. Al-Dujaili, Y. Duan, O. Al-Shamma, J. Santamaría, M. A. Fadhel, M. Al-Amidie, and L. Farhan, "Review of deep learning: concepts, cnn architectures, challenges, applications, future directions," *Journal of Big Data*, vol. 8, no. 1, Mar. 2021. [Online]. Available: <http://dx.doi.org/10.1186/s40537-021-00444-8>
- [19] J. Rose, A. El Gamal, and A. Sangiovanni-Vincentelli, "Architecture of field-programmable gate arrays," *Proceedings of the IEEE*, vol. 81, no. 7, pp. 1013–1029, 1993.
- [20] K. Zeng, Q. Ma, J. W. Wu, Z. Chen, T. Shen, and C. Yan, "Fpga-based accelerator for object detection: a comprehensive survey," *The Journal of Supercomputing*, vol. 78, no. 12, p. 14096–14136, Mar. 2022. [Online]. Available: <http://dx.doi.org/10.1007/s11227-022-04415-5>
- [21] J. M. Cardoso, J. G. F. Coutinho, and P. C. Diniz, "Chapter 2 - high-performance embedded computing," in *Embedded Computing for High Performance*, J. M. Cardoso, J. G. F. Coutinho, and P. C. Diniz, Eds. Boston: Morgan Kaufmann, 2017, pp. 17–56. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/B9780128041895000028>

- [22] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, *Key Metrics and Design Objectives*. Cham: Springer International Publishing, 2020, pp. 43–58. [Online]. Available: https://doi.org/10.1007/978-3-031-01766-7_3
- [23] ———, *Kernel Computation*. Cham: Springer International Publishing, 2020, pp. 59–72. [Online]. Available: https://doi.org/10.1007/978-3-031-01766-7_4
- [24] ———, *Reducing Precision*. Cham: Springer International Publishing, 2020, pp. 147–166. [Online]. Available: https://doi.org/10.1007/978-3-031-01766-7_7
- [25] H. Wu, P. Judd, X. Zhang, M. Isaev, and P. Micikevicius, “Integer quantization for deep learning inference: Principles and empirical evaluation,” 2020. [Online]. Available: <https://arxiv.org/abs/2004.09602>
- [26] X. Lin, C. Zhao, and W. Pan, “Towards accurate binary convolutional neural network,” 2017. [Online]. Available: <https://arxiv.org/abs/1711.11294>
- [27] C. Yuan and S. S. Agaian, “A comprehensive review of binary neural network,” *Artificial Intelligence Review*, vol. 56, no. 11, p. 12949–13013, Mar. 2023. [Online]. Available: <http://dx.doi.org/10.1007/s10462-023-10464-w>
- [28] Y. Chen, Y. Xie, L. Song, F. Chen, and T. Tang, “A survey of accelerator architectures for deep neural networks,” *Engineering*, vol. 6, no. 3, pp. 264–274, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2095809919306356>
- [29] Y.-H. Chen, J. Emer, and V. Sze, “Eyeriss: a spatial architecture for energy-efficient dataflow for convolutional neural networks,” *SIGARCH Comput. Archit. News*, vol. 44, no. 3, p. 367–379, Jun. 2016. [Online]. Available: <https://doi.org/10.1145/3007787.3001177>
- [30] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, *Designing DNN Accelerators*. Cham: Springer International Publishing, 2020, pp. 73–118. [Online]. Available: https://doi.org/10.1007/978-3-031-01766-7_5
- [31] N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers, R. Boyle, P. Iuc Cantin, C. Chao, C. Clark, J. Coriell, M. Daley, M. Dau, J. Dean, B. Gelb, T. V. Ghaemmamghami, R. Gottipati, W. Gulland, R. Hagmann, C. R. Ho, D. Hogberg, J. Hu, R. Hundt, D. Hurt, J. Ibarz, A. Jaffey, A. Jaworski, A. Kaplan, H. Khaitan, A. Koch, N. Kumar, S. Lacy, J. Laudon, J. Law, D. Le, C. Leary, Z. Liu, K. Lucke, A. Lundin, G. MacKean, A. Maggiore, M. Mahony, K. Miller, R. Nagarajan, R. Narayanaswami, R. Ni, K. Nix, T. Norrie, M. Omernick, N. Penukonda, A. Phelps, J. Ross, M. Ross, A. Salek, E. Samadiani, C. Severn, G. Sizikov, M. Snellman, J. Souter, D. Steinberg, A. Swing, M. Tan, G. Thorson, B. Tian, H. Toma, E. Tuttle, V. Vasudevan, R. Walter, W. Wang, E. Wilcox, and D. H. Yoon, “In-datacenter performance analysis of a tensor processing unit,” 2017. [Online]. Available: <https://arxiv.org/abs/1704.04760>

- [32] Y. Chen, T. Chen, Z. Xu, N. Sun, and O. Temam, “Diannao family: energy-efficient hardware accelerators for machine learning,” *Commun. ACM*, vol. 59, no. 11, p. 105–112, Oct. 2016. [Online]. Available: <https://doi.org/10.1145/2996864>
- [33] A. Parashar, M. Rhu, A. Mukkara, A. Puglielli, R. Venkatesan, B. Khailany, J. Emer, S. W. Keckler, and W. J. Dally, “Scnn: An accelerator for compressed-sparse convolutional neural networks,” 2017. [Online]. Available: <https://arxiv.org/abs/1708.04485>
- [34] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, “Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks,” *IEEE Journal of Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, 2017.
- [35] A. Raha, D. A. Mathaikutty, S. K. Ghosh, and S. Kundu, “Flexnn: A dataflow-aware flexible deep learning accelerator for energy-efficient edge devices,” 2024. [Online]. Available: <https://arxiv.org/abs/2403.09026>
- [36] V. Sze, Y.-H. Chen, T.-J. Yang, and J. Emer, “Efficient processing of deep neural networks: A tutorial and survey,” *Proceedings of the IEEE*, 2017.
- [37] NVDLA GitHub. [Online]. Available: <https://github.com/nvdla>
- [38] Y. Jia, E. Shelhamer, J. Donahue, S. Karayev, J. Long, R. Girshick, S. Guadarrama, and T. Darrell, “Caffe: Convolutional architecture for fast feature embedding,” *arXiv preprint arXiv:1408.5093*, 2014.
- [39] Y. Lai and C. Huang, “Object detection inference system with deep learning nvdla accelerator,” in *2022 IEEE International Conference on Consumer Electronics - Taiwan*, 2022, pp. 67–68.
- [40] S. Feng, J. Wu, S. Zhou, and R. Li, “The implementation of lenet-5 with nvdla on risc-v soc,” in *2019 IEEE 10th International Conference on Software Engineering and Service Science (ICSESS)*, 2019, pp. 39–42.
- [41] M. Meena, D. Vaithyanathan, P. Verma, and B. Kaur, “Hardware analysis on nvdla using resnet50,” in *2024 International Conference on Advances in Modern Age Technologies for Health and Engineering Science (AMATHE)*, 2024, pp. 1–5.
- [42] AMD, Zynq UltraScale+ MPSoC Data Sheet: Overview. [Online]. Available: https://www.amd.com/content/dam/xilinx/support/documents/data_sheets/ds891-zynq-ultrascale-plus-overview.pdf
- [43] R. Barbedo, “vivado-git,” 2024. [Online]. Available: <https://github.com/barbedo/vivado-git>

Appendix A

Example of operation

A.1 Xilinx NVDLA Wrapper

```
1 `include "commons.vh"
2
3 module xilNvDlaWrapper(
4     input m_axi_clk, // core_clk
5     input csb_clk,
6     input rstn,
7     input csb_rstn,
8
9     output dla_intr,
10    // dbb AXI
11    (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 m_axi AWVALID" *)
12    output nvdla_core2dbb_aw_awvalid,
13    (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 m_axi AWREADY" *)
14    input nvdla_core2dbb_aw_awready,
15    (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 m_axi AWID" *)
16    output [7:0] nvdla_core2dbb_aw_awid,
17    (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 m_axi AWLEN" *)
18    output [3:0] nvdla_core2dbb_aw_awlen,
19    (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 m_axi AWSIZE" *)
20    output [2:0] nvdla_core2dbb_aw_awsz,
21    (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 m_axi AWADDR" *)
22    output [64 -1:0] nvdla_core2dbb_aw_awaddr,
23    (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 m_axi WVALID" *)
24    output nvdla_core2dbb_w_wvalid,
25    (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 m_axi WREADY" *)
26    input nvdla_core2dbb_w_wready,
27    (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 m_axi WDATA" *)
28    output [64 -1:0] nvdla_core2dbb_w_wdata,
29    (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 m_axi WSTRB" *)
30    output [64/8-1:0] nvdla_core2dbb_w_wstrb,
31    (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 m_axi WLAST" *)
32    output nvdla_core2dbb_w_wlast,
```



```

33 (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 m_axi ARVALID" *)
34 output nvdla_core2dbb_ar_arvalid,
35 (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 m_axi ARREADY" *)
36 input nvdla_core2dbb_ar_arready,
37 (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 m_axi ARID" *)
38 output [7:0] nvdla_core2dbb_ar_arid,
39 (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 m_axi ARLEN" *)
40 output [3:0] nvdla_core2dbb_ar_arlen,
41 (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 m_axi ARSIZE" *)
42 output [2:0] nvdla_core2dbb_ar_arsize,
43 (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 m_axi ARADDR" *)
44 output [64 -1:0] nvdla_core2dbb_ar_araddr,
45 (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 m_axi BVALID" *)
46 input nvdla_core2dbb_b_bvalid,
47 (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 m_axi BREADY" *)
48 output nvdla_core2dbb_b_bready,
49 (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 m_axi BID" *)
50 input [7:0] nvdla_core2dbb_b_bid,
51 (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 m_axi RVALID" *)
52 input nvdla_core2dbb_r_rvalid,
53 (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 m_axi RREADY" *)
54 output nvdla_core2dbb_r_rready,
55 (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 m_axi RID" *)
56 input [7:0] nvdla_core2dbb_r_rid,
57 (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 m_axi RLAST" *)
58 input nvdla_core2dbb_r_rlast,
59 (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 m_axi RDATA" *)
60 input [64 -1:0] nvdla_core2dbb_r_rdata,
61 (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 m_axi AWBURST" *)
62 output [1:0] m_axi_awburst,
63 (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 m_axi AWLOCK" *)
64 output m_axi_awlock,
65 (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 m_axi AWCACHE" *)
66 output [3:0] m_axi_awcache,
67 (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 m_axi AWPROT" *)
68 output [2:0] m_axi_awprot,
69 (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 m_axi AWQOS" *)
70 output [3:0] m_axi_awqos,
71 (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 m_axi AWUSER" *)
72 output m_axi_awuser,
73 (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 m_axi BRESP" *)
74 input [1:0] m_axi_bresp,
75 (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 m_axi BUSER" *)
76 input m_axi_buser,
77 (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 m_axi ARBURST" *)
78 output [1:0] m_axi_arburst,
79 (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 m_axi ARLOCK" *)
80 output m_axi_arlock,
81 (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 m_axi ARCACHE" *)
82 output [3:0] m_axi_arcache,
83 (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 m_axi ARPROT" *)

```

```

84  output [2:0] m_axi_arprot,
85  (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 m_axi ARQOS" *)
86  output [3:0] m_axi_argos,
87  (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 m_axi ARUSER" *)
88  output m_axi_aruser,
89  (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 m_axi RRESP" *)
90  input [1:0] m_axi_rresp,
91  // cfg APB
92  (* X_INTERFACE_INFO = "xilinx.com:interface:apb:1.0 s_apb PSEL" *)
93  input psel,
94  (* X_INTERFACE_INFO = "xilinx.com:interface:apb:1.0 s_apb PENABLE" *)
95  input penable,
96  (* X_INTERFACE_INFO = "xilinx.com:interface:apb:1.0 s_apb PWRITE" *)
97  input pwrite,
98  (* X_INTERFACE_INFO = "xilinx.com:interface:apb:1.0 s_apb PADDR" *)
99  input [31:0] paddr,
100 (* X_INTERFACE_INFO = "xilinx.com:interface:apb:1.0 s_apb PWDATA" *)
101 input [31:0] pwrdata,
102 (* X_INTERFACE_INFO = "xilinx.com:interface:apb:1.0 s_apb PRDATA" *)
103 output [31:0] prdata,
104 (* X_INTERFACE_INFO = "xilinx.com:interface:apb:1.0 s_apb PREADY" *)
105 output pready,
106 (* X_INTERFACE_INFO = "xilinx.com:interface:apb:1.0 s_apb PSLVERR" *)
107 output pslverr
108 );
109
110 wire m_csb2nvdla_valid;
111 wire m_csb2nvdla_ready;
112 wire [15:0] m_csb2nvdla_addr;
113 wire [31:0] m_csb2nvdla_wdat;
114 wire m_csb2nvdla_write;
115 wire m_csb2nvdla_nposted;
116 wire m_nvdla2csb_valid;
117 wire [31:0] m_nvdla2csb_data;
118
119
120 NV_NVDLA_apb2csb apb2csb (
121     .pclk                (csb_clk)
122     ,.prstn              (csb_rstn)
123     ,.csb2nvdla_ready    (m_csb2nvdla_ready)
124     ,.nvdla2csb_data      (m_nvdla2csb_data)
125     ,.nvdla2csb_valid     (m_nvdla2csb_valid)
126     ,.paddr              (paddr)
127     ,.penable            (penable)
128     ,.psel               (psel)
129     ,.pwrdata            (pwrdata)
130     ,.pwrite             (pwrite)
131     ,.csb2nvdla_addr      (m_csb2nvdla_addr)
132     ,.csb2nvdla_nposted   (m_csb2nvdla_nposted)
133     ,.csb2nvdla_valid     (m_csb2nvdla_valid)
134     ,.csb2nvdla_wdat      (m_csb2nvdla_wdat)

```

```

135     ,.csb2nvdla_write      (m_csb2nvdla_write)
136     ,.prdata              (prdata)
137     ,.pready              (pready)
138 );
139
140
141 NV_nvdla nvdla_top (
142     .dla_core_clk          (m_axi_clk)
143     ,.dla_csb_clk          (csb_clk)
144     ,.global_clk_ovr_on    (1'b0)
145     ,.tmc2slcg_disable_clock_gating (1'b0)
146     ,.dla_reset_rstn       (rstn)
147     ,.direct_reset_        (1'b1)
148     ,.test_mode            (1'b0)
149     ,.csb2nvdla_valid      (m_csb2nvdla_valid)
150     ,.csb2nvdla_ready      (m_csb2nvdla_ready)
151     ,.csb2nvdla_addr       (m_csb2nvdla_addr)
152     ,.csb2nvdla_wdat       (m_csb2nvdla_wdat)
153     ,.csb2nvdla_write      (m_csb2nvdla_write)
154     ,.csb2nvdla_nposted    (m_csb2nvdla_nposted)
155     ,.nvdla2csb_valid      (m_nvdla2csb_valid)
156     ,.nvdla2csb_data       (m_nvdla2csb_data)
157     ,.nvdla2csb_wr_complete () //FIXME: no such port in apb2csb
158     ,.nvdla_core2dbb_aw_awvalid (nvdla_core2dbb_aw_awvalid)
159     ,.nvdla_core2dbb_aw_awready (nvdla_core2dbb_aw_awready)
160     ,.nvdla_core2dbb_aw_awaddr (nvdla_core2dbb_aw_awaddr)
161     ,.nvdla_core2dbb_aw_awid   (nvdla_core2dbb_aw_awid)
162     ,.nvdla_core2dbb_aw_awlen  (nvdla_core2dbb_aw_awlen)
163     ,.nvdla_core2dbb_w_wvalid  (nvdla_core2dbb_w_wvalid)
164     ,.nvdla_core2dbb_w_wready  (nvdla_core2dbb_w_wready)
165     ,.nvdla_core2dbb_w_wdata   (nvdla_core2dbb_w_wdata)
166     ,.nvdla_core2dbb_w_wstrb   (nvdla_core2dbb_w_wstrb)
167     ,.nvdla_core2dbb_w_wlast   (nvdla_core2dbb_w_wlast)
168     ,.nvdla_core2dbb_b_bvalid  (nvdla_core2dbb_b_bvalid)
169     ,.nvdla_core2dbb_b_bready  (nvdla_core2dbb_b_bready)
170     ,.nvdla_core2dbb_b_bid     (nvdla_core2dbb_b_bid)
171     ,.nvdla_core2dbb_ar_arvalid (nvdla_core2dbb_ar_arvalid)
172     ,.nvdla_core2dbb_ar_arready (nvdla_core2dbb_ar_arready)
173     ,.nvdla_core2dbb_ar_araddr (nvdla_core2dbb_ar_araddr)
174     ,.nvdla_core2dbb_ar_arid   (nvdla_core2dbb_ar_arid)
175     ,.nvdla_core2dbb_ar_arlen  (nvdla_core2dbb_ar_arlen)
176     ,.nvdla_core2dbb_r_rvalid  (nvdla_core2dbb_r_rvalid)
177     ,.nvdla_core2dbb_r_rready  (nvdla_core2dbb_r_rready)
178     ,.nvdla_core2dbb_r_rid     (nvdla_core2dbb_r_rid)
179     ,.nvdla_core2dbb_r_rlast   (nvdla_core2dbb_r_rlast)
180     ,.nvdla_core2dbb_r_rdata   (nvdla_core2dbb_r_rdata)
181     ,.dla_intr              (dla_intr)
182     ,.nvdla_pwrbus_ram_c_pd    (32'b0)
183     ,.nvdla_pwrbus_ram_ma_pd   (32'b0)
184     ,.nvdla_pwrbus_ram_mb_pd   (32'b0)
185     ,.nvdla_pwrbus_ram_p_pd    (32'b0)

```

```

186         ,.nvdla_pwrbus_ram_o_pd          (32'b0)
187         ,.nvdla_pwrbus_ram_a_pd          (32'b0)
188     ); // nvdla_top
189
190 assign nvdla_core2dbb_aw_awsiz = 3'b011;
191 assign nvdla_core2dbb_ar_arsiz = 3'b011;
192
193 assign m_axi_awburst = 2'b01;
194 assign m_axi_awlock = 1'b0;
195 assign m_axi_awcache = 4'b0010;
196 assign m_axi_awprot = 3'h0;
197 assign m_axi_awqos = 4'h0;
198 assign m_axi_awuser = 'b1;
199 assign m_axi_arburst = 2'b01;
200 assign m_axi_arlock = 1'b0;
201 assign m_axi_arcache = 4'b0010;
202 assign m_axi_arprot = 3'h0;
203 assign m_axi_arqos = 4'h0;
204 assign m_axi_aruser = 'b1;
205
206 assign pslverr = 1'b0;
207
208 endmodule

```

Listing A.1: Xilinx wrapper for NVDLA instance.

A.2 Tensor Reshape Functions

```

1
2 /*
3 * Change the tensor ordering from channel-last to 8B channel entries last
4 * following: https://nvdla.org/hw/format.html#feature-data-format
5 */
6 void Test_Utills_ReshapeTensorNVDLA(Tensor *t) {
7     Tensor temp;
8     tensor_clone(&temp, t);
9
10    int line_stride = t->width_n*ATOMIC_C;
11    int surface_stride = t->width_n*t->height_n*ATOMIC_C;
12    int surface_count = Xil_Ceil((float)t->channel_n / ATOMIC_C);
13
14    for(int k = 0; k < t->batch_n; k++) {
15        for(int r = 0; r < t->height_n; r++) {
16            for(int c = 0; c < t->width_n; c++) {
17                for(int ch = 0; ch < t->channel_n; ch++) {
18                    int8_t value = tensor_get(&temp, k, r, c, ch);
19
20                    int surface_idx = ch / ATOMIC_C;
21                    int atom_idx = ch % ATOMIC_C;

```

```

22
23         int idx = k * surface_count * surface_stride +
24                 surface_idx * surface_stride +
25                 r * line_stride +
26                 c * ATOMIC_C +
27                 atom_idx;
28         t->array[idx] = value;
29     }
30 }
31 }
32 }
33 }
34
35 /*
36  * Reshape the NVDLA tensor into channel-last ordering
37  */
38 void Test_Utills_ReshapeNVDLATensor(Tensor *t) {
39     Tensor temp;
40     tensor_clone(&temp, t);
41
42     int line_stride = t->width_n*ATOMIC_C;
43     int surface_stride = t->width_n*t->height_n*ATOMIC_C;
44     int surface_count = Xil_Ceil((float)t->channel_n / ATOMIC_C);
45
46     for(int k = 0; k < t->batch_n; k++) {
47         for(int r = 0; r < t->height_n; r++) {
48             for(int c = 0; c < t->width_n; c++) {
49                 for(int ch = 0; ch < t->channel_n; ch++) {
50                     int surface_idx = ch / ATOMIC_C;
51                     int atom_idx = ch % ATOMIC_C;
52
53                     int idx = k * surface_count * surface_stride +
54                             surface_idx * surface_stride +
55                             r * line_stride +
56                             c * ATOMIC_C +
57                             atom_idx;
58                     int8_t value = temp.array[idx];
59                     tensor_set(t, k, r, c, ch, value);
60
61                 }
62             }
63         }
64     }
65 }

```

Listing A.2: Tensor reshape functions. Atomic-C=8.

A.3 NVDLA Register Offsets

```

1 #ifndef NVDLA_REGISTER_OFFSETS_H
2 #define NVDLA_REGISTER_OFFSETS_H
3
4 #define NVDLA_HW_VERSION 0x0000
5 //GLB
6 #define NVDLA_GLB_S_INTR_MASK 0x1004
7 #define NVDLA_GLB_S_INTR_SET 0x1008
8 #define NVDLA_GLB_S_INTR_STATUS 0x100c
9 //MCIF
10 #define NVDLA_MCIF_CFG_RD_WEIGHT_0 0x2000
11 #define NVDLA_MCIF_CFG_RD_WEIGHT_1 0x2004
12 #define NVDLA_MCIF_CFG_RD_WEIGHT_2 0x2008
13 #define NVDLA_MCIF_CFG_WR_WEIGHT_0 0x200c
14 #define NVDLA_MCIF_CFG_WR_WEIGHT_1 0x2010
15 #define NVDLA_MCIF_CFG_OUTSTANDING_CNT 0x2014
16 #define NVDLA_MCIF_STATUS 0x2018
17 //CDMA
18 #define NVDLA_CDMA_S_STATUS 0x3000
19 #define NVDLA_CDMA_S_POINTER 0x3004
20 #define NVDLA_CDMA_S_ARBITER 0x3008
21 #define NVDLA_CDMA_S_CBUF_FLUSH_STATUS 0x300c
22 #define NVDLA_CDMA_D_OP_ENABLE 0x3010
23 #define NVDLA_CDMA_D_MISC_CFG 0x3014
24 #define NVDLA_CDMA_D_DATAIN_FORMAT 0x3018
25 #define NVDLA_CDMA_D_DATAIN_SIZE_0 0x301c
26 #define NVDLA_CDMA_D_DATAIN_SIZE_1 0x3020
27 #define NVDLA_CDMA_D_DATAIN_SIZE_EXT_0 0x3024
28 #define NVDLA_CDMA_D_PIXEL_OFFSET 0x3028
29 #define NVDLA_CDMA_D_DAIN_RAM_TYPE 0x302c
30 #define NVDLA_CDMA_D_DAIN_ADDR_HIGH_0 0x3030
31 #define NVDLA_CDMA_D_DAIN_ADDR_LOW_0 0x3034
32 #define NVDLA_CDMA_D_DAIN_ADDR_HIGH_1 0x3038
33 #define NVDLA_CDMA_D_DAIN_ADDR_LOW_1 0x303c
34 #define NVDLA_CDMA_D_LINE_STRIDE 0x3040
35 #define NVDLA_CDMA_D_LINE_UV_STRIDE 0x3044
36 #define NVDLA_CDMA_D_SURF_STRIDE 0x3048
37 #define NVDLA_CDMA_D_DAIN_MAP 0x304c
38 #define NVDLA_CDMA_D_BATCH_NUMBER 0x3058
39 #define NVDLA_CDMA_D_BATCH_STRIDE 0x305c
40 #define NVDLA_CDMA_D_ENTRY_PER_SLICE 0x3060
41 #define NVDLA_CDMA_D_FETCH_GRAIN 0x3064
42 #define NVDLA_CDMA_D_WEIGHT_FORMAT 0x3068
43 #define NVDLA_CDMA_D_WEIGHT_SIZE_0 0x306c
44 #define NVDLA_CDMA_D_WEIGHT_SIZE_1 0x3070
45 #define NVDLA_CDMA_D_WEIGHT_RAM_TYPE 0x3074
46 #define NVDLA_CDMA_D_WEIGHT_ADDR_HIGH 0x3078
47 #define NVDLA_CDMA_D_WEIGHT_ADDR_LOW 0x307c
48 #define NVDLA_CDMA_D_WEIGHT_BYTES 0x3080
49 #define NVDLA_CDMA_D_WGS_ADDR_HIGH 0x3084
50 #define NVDLA_CDMA_D_WGS_ADDR_LOW 0x3088
51 #define NVDLA_CDMA_D_WMB_ADDR_HIGH 0x308c

```

```

52 #define NVDLA_CDMA_D_WMB_ADDR_LOW 0x3090
53 #define NVDLA_CDMA_D_WMB_BYTES 0x3094
54 #define NVDLA_CDMA_D_MEAN_FORMAT 0x3098
55 #define NVDLA_CDMA_D_MEAN_GLOBAL_0 0x309c
56 #define NVDLA_CDMA_D_MEAN_GLOBAL_1 0x30a0
57 #define NVDLA_CDMA_D_CVT_CFG 0x30a4
58 #define NVDLA_CDMA_D_CVT_OFFSET 0x30a8
59 #define NVDLA_CDMA_D_CVT_SCALE 0x30ac
60 #define NVDLA_CDMA_D_CONV_STRIDE 0x30b0
61 #define NVDLA_CDMA_D_ZERO_PADDING 0x30b4
62 #define NVDLA_CDMA_D_ZERO_PADDING_VALUE 0x30b8
63 #define NVDLA_CDMA_D_BANK 0x30bc
64 #define NVDLA_CDMA_D_NAN_FLUSH_TO_ZERO 0x30c0
65 #define NVDLA_CDMA_D_NAN_INPUT_DATA_NUM 0x30c4
66 #define NVDLA_CDMA_D_NAN_INPUT_WEIGHT_NUM 0x30c8
67 #define NVDLA_CDMA_D_INF_INPUT_DATA_NUM 0x30cc
68 #define NVDLA_CDMA_D_INF_INPUT_WEIGHT_NUM 0x30d0
69 #define NVDLA_CDMA_D_PERF_ENABLE 0x30d4
70 #define NVDLA_CDMA_D_PERF_DAT_READ_STALL 0x30d8
71 #define NVDLA_CDMA_D_PERF_WT_READ_STALL 0x30dc
72 #define NVDLA_CDMA_D_PERF_DAT_READ_LATENCY 0x30e0
73 #define NVDLA_CDMA_D_PERF_WT_READ_LATENCY 0x30e4
74 //CSC
75 #define NVDLA_CSC_S_STATUS 0x4000
76 #define NVDLA_CSC_S_POINTER 0x4004
77 #define NVDLA_CSC_D_OP_ENABLE 0x4008
78 #define NVDLA_CSC_D_MISC_CFG 0x400c
79 #define NVDLA_CSC_D_DATAIN_FORMAT 0x4010
80 #define NVDLA_CSC_D_DATAIN_SIZE_EXT_0 0x4014
81 #define NVDLA_CSC_D_DATAIN_SIZE_EXT_1 0x4018
82 #define NVDLA_CSC_D_BATCH_NUMBER 0x401c
83 #define NVDLA_CSC_D_POST_Y_EXTENSION 0x4020
84 #define NVDLA_CSC_D_ENTRY_PER_SLICE 0x4024
85 #define NVDLA_CSC_D_WEIGHT_FORMAT 0x4028
86 #define NVDLA_CSC_D_WEIGHT_SIZE_EXT_0 0x402c
87 #define NVDLA_CSC_D_WEIGHT_SIZE_EXT_1 0x4030
88 #define NVDLA_CSC_D_WEIGHT_BYTES 0x4034
89 #define NVDLA_CSC_D_WMB_BYTES 0x4038
90 #define NVDLA_CSC_D_DATAOUT_SIZE_0 0x403c
91 #define NVDLA_CSC_D_DATAOUT_SIZE_1 0x4040
92 #define NVDLA_CSC_D_ATOMICS 0x4044
93 #define NVDLA_CSC_D_RELEASE 0x4048
94 #define NVDLA_CSC_D_CONV_STRIDE_EXT 0x404c
95 #define NVDLA_CSC_D_DILATION_EXT 0x4050
96 #define NVDLA_CSC_D_ZERO_PADDING 0x4054
97 #define NVDLA_CSC_D_ZERO_PADDING_VALUE 0x4058
98 #define NVDLA_CSC_D_BANK 0x405c
99 #define NVDLA_CSC_D_PRA_CFG 0x4060
100 //CMAC_A
101 #define NVDLA_CMAC_A_S_STATUS 0x5000
102 #define NVDLA_CMAC_A_S_POINTER 0x5004

```

```

103 #define NVDLA_CMACE_A_D_OP_ENABLE 0x5008
104 #define NVDLA_CMACE_A_D_MISC_CFG 0x500c
105 //CMACE_B
106 #define NVDLA_CMACE_B_S_STATUS 0x6000
107 #define NVDLA_CMACE_B_S_POINTER 0x6004
108 #define NVDLA_CMACE_B_D_OP_ENABLE 0x6008
109 #define NVDLA_CMACE_B_D_MISC_CFG 0x600c
110 //CACC
111 #define NVDLA_CACC_S_STATUS 0x7000
112 #define NVDLA_CACC_S_POINTER 0x7004
113 #define NVDLA_CACC_D_OP_ENABLE 0x7008
114 #define NVDLA_CACC_D_MISC_CFG 0x700c
115 #define NVDLA_CACC_D_DATAOUT_SIZE_0 0x7010
116 #define NVDLA_CACC_D_DATAOUT_SIZE_1 0x7014
117 #define NVDLA_CACC_D_DATAOUT_ADDR 0x7018
118 #define NVDLA_CACC_D_BATCH_NUMBER 0x701c
119 #define NVDLA_CACC_D_LINE_STRIDE 0x7020
120 #define NVDLA_CACC_D_SURF_STRIDE 0x7024
121 #define NVDLA_CACC_D_DATAOUT_MAP 0x7028
122 #define NVDLA_CACC_D_CLIP_CFG 0x702c
123 #define NVDLA_CACC_D_OUT_SATURATION 0x7030
124 //SDP_RDMA
125 #define NVDLA_SDP_RDMA_S_STATUS 0x8000
126 #define NVDLA_SDP_RDMA_S_POINTER 0x8004
127 #define NVDLA_SDP_RDMA_D_OP_ENABLE 0x8008
128 #define NVDLA_SDP_RDMA_D_DATA_CUBE_WIDTH 0x800c
129 #define NVDLA_SDP_RDMA_D_DATA_CUBE_HEIGHT 0x8010
130 #define NVDLA_SDP_RDMA_D_DATA_CUBE_CHANNEL 0x8014
131 #define NVDLA_SDP_RDMA_D_SRC_BASE_ADDR_LOW 0x8018
132 #define NVDLA_SDP_RDMA_D_SRC_BASE_ADDR_HIGH 0x801c
133 #define NVDLA_SDP_RDMA_D_SRC_LINE_STRIDE 0x8020
134 #define NVDLA_SDP_RDMA_D_SRC_SURFACE_STRIDE 0x8024
135 #define NVDLA_SDP_RDMA_D_BRDMA_CFG 0x8028
136 #define NVDLA_SDP_RDMA_D_BS_BASE_ADDR_LOW 0x802c
137 #define NVDLA_SDP_RDMA_D_BS_BASE_ADDR_HIGH 0x8030
138 #define NVDLA_SDP_RDMA_D_BS_LINE_STRIDE 0x8034
139 #define NVDLA_SDP_RDMA_D_BS_SURFACE_STRIDE 0x8038
140 #define NVDLA_SDP_RDMA_D_BS_BATCH_STRIDE 0x803c
141 #define NVDLA_SDP_RDMA_D_NRDMA_CFG 0x8040
142 #define NVDLA_SDP_RDMA_D_BN_BASE_ADDR_LOW 0x8044
143 #define NVDLA_SDP_RDMA_D_BN_BASE_ADDR_HIGH 0x8048
144 #define NVDLA_SDP_RDMA_D_BN_LINE_STRIDE 0x804c
145 #define NVDLA_SDP_RDMA_D_BN_SURFACE_STRIDE 0x8050
146 #define NVDLA_SDP_RDMA_D_BN_BATCH_STRIDE 0x8054
147 #define NVDLA_SDP_RDMA_D_ERDMA_CFG 0x8058
148 #define NVDLA_SDP_RDMA_D_EW_BASE_ADDR_LOW 0x805c
149 #define NVDLA_SDP_RDMA_D_EW_BASE_ADDR_HIGH 0x8060
150 #define NVDLA_SDP_RDMA_D_EW_LINE_STRIDE 0x8064
151 #define NVDLA_SDP_RDMA_D_EW_SURFACE_STRIDE 0x8068
152 #define NVDLA_SDP_RDMA_D_EW_BATCH_STRIDE 0x806c
153 #define NVDLA_SDP_RDMA_D_FEATURE_MODE_CFG 0x8070

```



```

154 #define NVDLA_SDP_RDMA_D_SRC_DMA_CFG 0x8074
155 #define NVDLA_SDP_RDMA_D_STATUS_NAN_INPUT_NUM 0x8078
156 #define NVDLA_SDP_RDMA_D_STATUS_INF_INPUT_NUM 0x807c
157 #define NVDLA_SDP_RDMA_D_PERF_ENABLE 0x8080
158 #define NVDLA_SDP_RDMA_D_PERF_MRDMA_READ_STALL 0x8084
159 #define NVDLA_SDP_RDMA_D_PERF_BRDMA_READ_STALL 0x8088
160 #define NVDLA_SDP_RDMA_D_PERF_NRDMA_READ_STALL 0x808c
161 #define NVDLA_SDP_RDMA_D_PERF_ERDMA_READ_STALL 0x8090
162 //SDP
163 #define NVDLA_SDP_S_STATUS 0x9000
164 #define NVDLA_SDP_S_POINTER 0x9004
165 #define NVDLA_SDP_S_LUT_ACCESS_CFG 0x9008
166 #define NVDLA_SDP_S_LUT_ACCESS_DATA 0x900c
167 #define NVDLA_SDP_S_LUT_CFG 0x9010
168 #define NVDLA_SDP_S_LUT_INFO 0x9014
169 #define NVDLA_SDP_S_LUT_LE_START 0x9018
170 #define NVDLA_SDP_S_LUT_LE_END 0x901c
171 #define NVDLA_SDP_S_LUT_LO_START 0x9020
172 #define NVDLA_SDP_S_LUT_LO_END 0x9024
173 #define NVDLA_SDP_S_LUT_LE_SLOPE_SCALE 0x9028
174 #define NVDLA_SDP_S_LUT_LE_SLOPE_SHIFT 0x902c
175 #define NVDLA_SDP_S_LUT_LO_SLOPE_SCALE 0x9030
176 #define NVDLA_SDP_S_LUT_LO_SLOPE_SHIFT 0x9034
177 #define NVDLA_SDP_D_OP_ENABLE 0x9038
178 #define NVDLA_SDP_D_DATA_CUBE_WIDTH 0x903c
179 #define NVDLA_SDP_D_DATA_CUBE_HEIGHT 0x9040
180 #define NVDLA_SDP_D_DATA_CUBE_CHANNEL 0x9044
181 #define NVDLA_SDP_D_DST_BASE_ADDR_LOW 0x9048
182 #define NVDLA_SDP_D_DST_BASE_ADDR_HIGH 0x904c
183 #define NVDLA_SDP_D_DST_LINE_STRIDE 0x9050
184 #define NVDLA_SDP_D_DST_SURFACE_STRIDE 0x9054
185 #define NVDLA_SDP_D_DP_BS_CFG 0x9058
186 #define NVDLA_SDP_D_DP_BS_ALU_CFG 0x905c
187 #define NVDLA_SDP_D_DP_BS_ALU_SRC_VALUE 0x9060
188 #define NVDLA_SDP_D_DP_BS_MUL_CFG 0x9064
189 #define NVDLA_SDP_D_DP_BS_MUL_SRC_VALUE 0x9068
190 #define NVDLA_SDP_D_DP_BN_CFG 0x906c
191 #define NVDLA_SDP_D_DP_BN_ALU_CFG 0x9070
192 #define NVDLA_SDP_D_DP_BN_ALU_SRC_VALUE 0x9074
193 #define NVDLA_SDP_D_DP_BN_MUL_CFG 0x9078
194 #define NVDLA_SDP_D_DP_BN_MUL_SRC_VALUE 0x907c
195 #define NVDLA_SDP_D_DP_EW_CFG 0x9080
196 #define NVDLA_SDP_D_DP_EW_ALU_CFG 0x9084
197 #define NVDLA_SDP_D_DP_EW_ALU_SRC_VALUE 0x9088
198 #define NVDLA_SDP_D_DP_EW_ALU_CVT_OFFSET_VALUE 0x908c
199 #define NVDLA_SDP_D_DP_EW_ALU_CVT_SCALE_VALUE 0x9090
200 #define NVDLA_SDP_D_DP_EW_ALU_CVT_TRUNCATE_VALUE 0x9094
201 #define NVDLA_SDP_D_DP_EW_MUL_CFG 0x9098
202 #define NVDLA_SDP_D_DP_EW_MUL_SRC_VALUE 0x909c
203 #define NVDLA_SDP_D_DP_EW_MUL_CVT_OFFSET_VALUE 0x90a0
204 #define NVDLA_SDP_D_DP_EW_MUL_CVT_SCALE_VALUE 0x90a4

```

```

205 #define NVDLA_SDP_D_DP_EW_MUL_CVT_TRUNCATE_VALUE 0x90a8
206 #define NVDLA_SDP_D_DP_EW_TRUNCATE_VALUE 0x90ac
207 #define NVDLA_SDP_D_FEATURE_MODE_CFG 0x90b0
208 #define NVDLA_SDP_D_DST_DMA_CFG 0x90b4
209 #define NVDLA_SDP_D_DST_BATCH_STRIDE 0x90b8
210 #define NVDLA_SDP_D_DATA_FORMAT 0x90bc
211 #define NVDLA_SDP_D_CVT_OFFSET 0x90c0
212 #define NVDLA_SDP_D_CVT_SCALE 0x90c4
213 #define NVDLA_SDP_D_CVT_SHIFT 0x90c8
214 #define NVDLA_SDP_D_STATUS 0x90cc
215 #define NVDLA_SDP_D_STATUS_NAN_INPUT_NUM 0x90d0
216 #define NVDLA_SDP_D_STATUS_INF_INPUT_NUM 0x90d4
217 #define NVDLA_SDP_D_STATUS_NAN_OUTPUT_NUM 0x90d8
218 #define NVDLA_SDP_D_PERF_ENABLE 0x90dc
219 #define NVDLA_SDP_D_PERF_WDMA_WRITE_STALL 0x90e0
220 #define NVDLA_SDP_D_PERF_LUT_UFLOW 0x90e4
221 #define NVDLA_SDP_D_PERF_LUT_OFLOW 0x90e8
222 #define NVDLA_SDP_D_PERF_OUT_SATURATION 0x90ec
223 #define NVDLA_SDP_D_PERF_LUT_HYBRID 0x90f0
224 #define NVDLA_SDP_D_PERF_LUT_LE_HIT 0x90f4
225 #define NVDLA_SDP_D_PERF_LUT_LO_HIT 0x90f8
226 //PDP_RDMA
227 #define NVDLA_PDP_RDMA_S_STATUS 0xa000
228 #define NVDLA_PDP_RDMA_S_POINTER 0xa004
229 #define NVDLA_PDP_RDMA_D_OP_ENABLE 0xa008
230 #define NVDLA_PDP_RDMA_D_DATA_CUBE_IN_WIDTH 0xa00c
231 #define NVDLA_PDP_RDMA_D_DATA_CUBE_IN_HEIGHT 0xa010
232 #define NVDLA_PDP_RDMA_D_DATA_CUBE_IN_CHANNEL 0xa014
233 #define NVDLA_PDP_RDMA_D_FLYING_MODE 0xa018
234 #define NVDLA_PDP_RDMA_D_SRC_BASE_ADDR_LOW 0xa01c
235 #define NVDLA_PDP_RDMA_D_SRC_BASE_ADDR_HIGH 0xa020
236 #define NVDLA_PDP_RDMA_D_SRC_LINE_STRIDE 0xa024
237 #define NVDLA_PDP_RDMA_D_SRC_SURFACE_STRIDE 0xa028
238 #define NVDLA_PDP_RDMA_D_SRC_RAM_CFG 0xa02c
239 #define NVDLA_PDP_RDMA_D_DATA_FORMAT 0xa030
240 #define NVDLA_PDP_RDMA_D_OPERATION_MODE_CFG 0xa034
241 #define NVDLA_PDP_RDMA_D_POOLING_KERNEL_CFG 0xa038
242 #define NVDLA_PDP_RDMA_D_POOLING_PADDING_CFG 0xa03c
243 #define NVDLA_PDP_RDMA_D_PARTIAL_WIDTH_IN 0xa040
244 #define NVDLA_PDP_RDMA_D_PERF_ENABLE 0xa044
245 #define NVDLA_PDP_RDMA_D_PERF_READ_STALL 0xa048
246 //PDP
247 #define NVDLA_PDP_S_STATUS 0xb000
248 #define NVDLA_PDP_S_POINTER 0xb004
249 #define NVDLA_PDP_D_OP_ENABLE 0xb008
250 #define NVDLA_PDP_D_DATA_CUBE_IN_WIDTH 0xb00c
251 #define NVDLA_PDP_D_DATA_CUBE_IN_HEIGHT 0xb010
252 #define NVDLA_PDP_D_DATA_CUBE_IN_CHANNEL 0xb014
253 #define NVDLA_PDP_D_DATA_CUBE_OUT_WIDTH 0xb018
254 #define NVDLA_PDP_D_DATA_CUBE_OUT_HEIGHT 0xb01c
255 #define NVDLA_PDP_D_DATA_CUBE_OUT_CHANNEL 0xb020

```

```

256 #define NVDLA_PDP_D_OPERATION_MODE_CFG 0xb024
257 #define NVDLA_PDP_D_NAN_FLUSH_TO_ZERO 0xb028
258 #define NVDLA_PDP_D_PARTIAL_WIDTH_IN 0xb02c
259 #define NVDLA_PDP_D_PARTIAL_WIDTH_OUT 0xb030
260 #define NVDLA_PDP_D_POOLING_KERNEL_CFG 0xb034
261 #define NVDLA_PDP_D_RECIP_KERNEL_WIDTH 0xb038
262 #define NVDLA_PDP_D_RECIP_KERNEL_HEIGHT 0xb03c
263 #define NVDLA_PDP_D_POOLING_PADDING_CFG 0xb040
264 #define NVDLA_PDP_D_POOLING_PADDING_VALUE_1_CFG 0xb044
265 #define NVDLA_PDP_D_POOLING_PADDING_VALUE_2_CFG 0xb048
266 #define NVDLA_PDP_D_POOLING_PADDING_VALUE_3_CFG 0xb04c
267 #define NVDLA_PDP_D_POOLING_PADDING_VALUE_4_CFG 0xb050
268 #define NVDLA_PDP_D_POOLING_PADDING_VALUE_5_CFG 0xb054
269 #define NVDLA_PDP_D_POOLING_PADDING_VALUE_6_CFG 0xb058
270 #define NVDLA_PDP_D_POOLING_PADDING_VALUE_7_CFG 0xb05c
271 #define NVDLA_PDP_D_SRC_BASE_ADDR_LOW 0xb060
272 #define NVDLA_PDP_D_SRC_BASE_ADDR_HIGH 0xb064
273 #define NVDLA_PDP_D_SRC_LINE_STRIDE 0xb068
274 #define NVDLA_PDP_D_SRC_SURFACE_STRIDE 0xb06c
275 #define NVDLA_PDP_D_DST_BASE_ADDR_LOW 0xb070
276 #define NVDLA_PDP_D_DST_BASE_ADDR_HIGH 0xb074
277 #define NVDLA_PDP_D_DST_LINE_STRIDE 0xb078
278 #define NVDLA_PDP_D_DST_SURFACE_STRIDE 0xb07c
279 #define NVDLA_PDP_D_DST_RAM_CFG 0xb080
280 #define NVDLA_PDP_D_DATA_FORMAT 0xb084
281 #define NVDLA_PDP_D_INF_INPUT_NUM 0xb088
282 #define NVDLA_PDP_D_NAN_INPUT_NUM 0xb08c
283 #define NVDLA_PDP_D_NAN_OUTPUT_NUM 0xb090
284 #define NVDLA_PDP_D_PERF_ENABLE 0xb094
285 #define NVDLA_PDP_D_PERF_WRITE_STALL 0xb098
286 //CDP_RDMA
287 #define NVDLA_CDP_RDMA_S_STATUS 0xc000
288 #define NVDLA_CDP_RDMA_S_POINTER 0xc004
289 #define NVDLA_CDP_RDMA_D_OP_ENABLE 0xc008
290 #define NVDLA_CDP_RDMA_D_DATA_CUBE_WIDTH 0xc00c
291 #define NVDLA_CDP_RDMA_D_DATA_CUBE_HEIGHT 0xc010
292 #define NVDLA_CDP_RDMA_D_DATA_CUBE_CHANNEL 0xc014
293 #define NVDLA_CDP_RDMA_D_SRC_BASE_ADDR_LOW 0xc018
294 #define NVDLA_CDP_RDMA_D_SRC_BASE_ADDR_HIGH 0xc01c
295 #define NVDLA_CDP_RDMA_D_SRC_LINE_STRIDE 0xc020
296 #define NVDLA_CDP_RDMA_D_SRC_SURFACE_STRIDE 0xc024
297 #define NVDLA_CDP_RDMA_D_SRC_DMA_CFG 0xc028
298 #define NVDLA_CDP_RDMA_D_SRC_COMPRESSION_EN 0xc02c
299 #define NVDLA_CDP_RDMA_D_OPERATION_MODE 0xc030
300 #define NVDLA_CDP_RDMA_D_DATA_FORMAT 0xc034
301 #define NVDLA_CDP_RDMA_D_PERF_ENABLE 0xc038
302 #define NVDLA_CDP_RDMA_D_PERF_READ_STALL 0xc03c
303 //CDP
304 #define NVDLA_CDP_S_STATUS 0xd000
305 #define NVDLA_CDP_S_POINTER 0xd004
306 #define NVDLA_CDP_S_LUT_ACCESS_CFG 0xd008

```

```

307 #define NVDLA_CDP_S_LUT_ACCESS_DATA 0xd00c
308 #define NVDLA_CDP_S_LUT_CFG 0xd010
309 #define NVDLA_CDP_S_LUT_INFO 0xd014
310 #define NVDLA_CDP_S_LUT_LE_START_LOW 0xd018
311 #define NVDLA_CDP_S_LUT_LE_START_HIGH 0xd01c
312 #define NVDLA_CDP_S_LUT_LE_END_LOW 0xd020
313 #define NVDLA_CDP_S_LUT_LE_END_HIGH 0xd024
314 #define NVDLA_CDP_S_LUT_LO_START_LOW 0xd028
315 #define NVDLA_CDP_S_LUT_LO_START_HIGH 0xd02c
316 #define NVDLA_CDP_S_LUT_LO_END_LOW 0xd030
317 #define NVDLA_CDP_S_LUT_LO_END_HIGH 0xd034
318 #define NVDLA_CDP_S_LUT_LE_SLOPE_SCALE 0xd038
319 #define NVDLA_CDP_S_LUT_LE_SLOPE_SHIFT 0xd03c
320 #define NVDLA_CDP_S_LUT_LO_SLOPE_SCALE 0xd040
321 #define NVDLA_CDP_S_LUT_LO_SLOPE_SHIFT 0xd044
322 #define NVDLA_CDP_D_OP_ENABLE 0xd048
323 #define NVDLA_CDP_D_FUNC_BYPASS 0xd04c
324 #define NVDLA_CDP_D_DST_BASE_ADDR_LOW 0xd050
325 #define NVDLA_CDP_D_DST_BASE_ADDR_HIGH 0xd054
326 #define NVDLA_CDP_D_DST_LINE_STRIDE 0xd058
327 #define NVDLA_CDP_D_DST_SURFACE_STRIDE 0xd05c
328 #define NVDLA_CDP_D_DST_DMA_CFG 0xd060
329 #define NVDLA_CDP_D_DST_COMPRESSION_EN 0xd064
330 #define NVDLA_CDP_D_DATA_FORMAT 0xd068
331 #define NVDLA_CDP_D_NAN_FLUSH_TO_ZERO 0xd06c
332 #define NVDLA_CDP_D_LRN_CFG 0xd070
333 #define NVDLA_CDP_D_DATIN_OFFSET 0xd074
334 #define NVDLA_CDP_D_DATIN_SCALE 0xd078
335 #define NVDLA_CDP_D_DATIN_SHIFTER 0xd07c
336 #define NVDLA_CDP_D_DATOUT_OFFSET 0xd080
337 #define NVDLA_CDP_D_DATOUT_SCALE 0xd084
338 #define NVDLA_CDP_D_DATOUT_SHIFTER 0xd088
339 #define NVDLA_CDP_D_NAN_INPUT_NUM 0xd08c
340 #define NVDLA_CDP_D_INF_INPUT_NUM 0xd090
341 #define NVDLA_CDP_D_NAN_OUTPUT_NUM 0xd094
342 #define NVDLA_CDP_D_OUT_SATURATION 0xd098
343 #define NVDLA_CDP_D_PERF_ENABLE 0xd09c
344 #define NVDLA_CDP_D_PERF_WRITE_STALL 0xd0a0
345 #define NVDLA_CDP_D_PERF_LUT_UFLOW 0xd0a4
346 #define NVDLA_CDP_D_PERF_LUT_OFLOW 0xd0a8
347 #define NVDLA_CDP_D_PERF_LUT_HYBRID 0xd0ac
348 #define NVDLA_CDP_D_PERF_LUT_LE_HIT 0xd0b0
349 #define NVDLA_CDP_D_PERF_LUT_LO_HIT 0xd0b4
350
351
352 #endif

```

Listing A.3: Software macros for NVDLA register offsets.

A.4 Stage SDP Passthrough NVDLA

```
1  /*
2  * Stage addresses to tensors and tensor attributes
3  * to SDP to perform pass-through operation
4  */
5  void NVDLA_Utills_StagePassThroughSDP(Tensor *src, Tensor *dst, int relu) {
6      nvdlaregWrite(NVDLA_SDP_D_OP_ENABLE, 0x0);
7      nvdlaregWrite(NVDLA_SDP_RDMA_D_OP_ENABLE, 0x0);
8
9      nvdlaregWrite(NVDLA_SDP_RDMA_S_POINTER, 0x0);
10     nvdlaregWrite(NVDLA_SDP_RDMA_S_STATUS, 0x0);
11     nvdlaregWrite(NVDLA_SDP_S_POINTER, 0x0);
12     nvdlaregWrite(NVDLA_SDP_S_STATUS, 0x0);
13     nvdlaregWrite(NVDLA_SDP_D_STATUS, 0x0);
14
15     nvdlaregWrite(NVDLA_SDP_RDMA_D_DATA_CUBE_WIDTH, src->width_n - 1);
16     nvdlaregWrite(NVDLA_SDP_RDMA_D_DATA_CUBE_HEIGHT, src->height_n - 1);
17     nvdlaregWrite(NVDLA_SDP_RDMA_D_DATA_CUBE_CHANNEL, src->channel_n - 1);
18
19     nvdlaregWrite(NVDLA_SDP_RDMA_D_SRC_LINE_STRIDE, src->width_n*ATOMIC_C);
20     nvdlaregWrite(NVDLA_SDP_RDMA_D_SRC_SURFACE_STRIDE, (src->width_n*src->
height_n*ATOMIC_C));
21
22     nvdlaregWrite(NVDLA_SDP_D_DATA_CUBE_WIDTH, src->width_n - 1);
23     nvdlaregWrite(NVDLA_SDP_D_DATA_CUBE_HEIGHT, src->height_n - 1);
24     nvdlaregWrite(NVDLA_SDP_D_DATA_CUBE_CHANNEL, src->channel_n - 1);
25
26     nvdlaregWrite(NVDLA_SDP_D_DST_LINE_STRIDE, src->width_n*ATOMIC_C);
27     nvdlaregWrite(NVDLA_SDP_D_DST_SURFACE_STRIDE, (src->width_n*src->height_n*
ATOMIC_C));
28
29
30     uint64_t src_addr = (uint64_t)src->array;
31     uint64_t dst_addr = (uint64_t)dst->array;
32
33     nvdlaregWrite(NVDLA_SDP_RDMA_D_SRC_BASE_ADDR_LOW, src_addr & 0xFFFFFFFF);
34     nvdlaregWrite(NVDLA_SDP_RDMA_D_SRC_BASE_ADDR_HIGH, (src_addr >> 32) & 0
FFFFFFFF);
35
36     nvdlaregWrite(NVDLA_SDP_D_DST_BASE_ADDR_LOW, dst_addr & 0xFFFFFFFF);
37     nvdlaregWrite(NVDLA_SDP_D_DST_BASE_ADDR_HIGH, (dst_addr >> 32) & 0
FFFFFFFF);
38     nvdlaregWrite(NVDLA_SDP_S_STATUS, 0);
39     nvdlaregWrite(NVDLA_SDP_S_POINTER, 0);
40     nvdlaregWrite(NVDLA_SDP_D_FEATURE_MODE_CFG, 8); // Read from RDMA core,
set NAN to 0
41
42     // Bypass ALU and MUL, keep RELU if needed
43     if(relu) {
```

```

44     nvdlaRegWrite(NVDLA_SDP_D_DP_BS_CFG, SDP_BS_CFG_ALU_BYPASS |
SDP_BS_CFG_MUL_BYPASS);
45 } else {
46     nvdlaRegWrite(NVDLA_SDP_D_DP_BS_CFG, SDP_BS_CFG_BYPASS);
47 }
48 nvdlaRegWrite(NVDLA_SDP_D_DP_BN_CFG, SDP_BN_CFG_BYPASS); // Disable BN
49 nvdlaRegWrite(NVDLA_SDP_D_DP_EW_CFG, SDP_EW_CFG_BYPASS); // Disable EW
50 nvdlaRegWrite(NVDLA_SDP_D_DST_DMA_CFG, 1);
51 nvdlaRegWrite(NVDLA_SDP_D_DATA_FORMAT, 0);
52 nvdlaRegWrite(NVDLA_SDP_D_CVT_OFFSET, 0);
53 nvdlaRegWrite(NVDLA_SDP_D_STATUS, 0);
54 nvdlaRegWrite(NVDLA_SDP_D_CVT_SHIFT, 0);
55 nvdlaRegWrite(NVDLA_SDP_D_CVT_SCALE, 1);
56
57
58 nvdlaRegWrite(NVDLA_SDP_D_DP_BN_CFG, SDP_BN_CFG_BYPASS);
59
60 nvdlaRegWrite(NVDLA_SDP_RDMA_D_FEATURE_MODE_CFG, 0x0);
61 nvdlaRegWrite(NVDLA_SDP_RDMA_D_BRDMA_CFG, 0x13); // Disable BRDMA
62 nvdlaRegWrite(NVDLA_SDP_RDMA_D_ERDMA_CFG, 0x3); // Disable ERDMA
63 nvdlaRegWrite(NVDLA_SDP_RDMA_D_NRDMA_CFG, 0xd); // Disable NRDMA
64 nvdlaRegWrite(NVDLA_SDP_RDMA_D_PERF_BRDMA_READ_STALL, 0x0);
65 nvdlaRegWrite(NVDLA_SDP_RDMA_D_PERF_ENABLE, 0x2);
66 nvdlaRegWrite(NVDLA_SDP_RDMA_D_PERF_ERDMA_READ_STALL, 0x0);
67 nvdlaRegWrite(NVDLA_SDP_RDMA_D_PERF_MRDMA_READ_STALL, 0x0);
68 nvdlaRegWrite(NVDLA_SDP_RDMA_D_PERF_NRDMA_READ_STALL, 0x0);
69 nvdlaRegWrite(NVDLA_SDP_RDMA_D_SRC_BASE_ADDR_HIGH, 0x0);
70 nvdlaRegWrite(NVDLA_SDP_RDMA_D_SRC_DMA_CFG, 0x1);
71 nvdlaRegWrite(NVDLA_SDP_RDMA_D_STATUS_INF_INPUT_NUM, 0x0);
72 nvdlaRegWrite(NVDLA_SDP_RDMA_D_STATUS_NAN_INPUT_NUM, 0x0);
73
74 }
75
76 #define RELU 1
77
78 int sdp_pass_through_tensor()
79 {
80
81     Tensor source_tensor = {
82         .batch_n = 1,
83         .height_n = 15,
84         .width_n = 20,
85         .channel_n = 1
86     };
87
88     Tensor dst_tensor = {
89         .batch_n = 1,
90         .height_n = 15,
91         .width_n = 20,
92         .channel_n = 1
93     };

```

```

94
95
96     int i = 0;
97     for(int k = 0; k < source_tensor.batch_n; k++) {
98         for(int r = 0; r < source_tensor.height_n; r++) {
99             for(int c = 0; c < source_tensor.width_n; c++) {
100                 for(int ch = 0; ch < source_tensor.channel_n; ch++) {
101 #if RELU
102                     tensor_set(&source_tensor, k, r, c, ch, (i % 2 == 0)? i :
103                         -i);
104 #else
105                     tensor_set(&source_tensor, k, r, c, ch, i);
106 #endif
107                     i++;
108                 }
109             }
110         }
111
112     for(int k = 0; k < dst_tensor.batch_n; k++) {
113         for(int r = 0; r < dst_tensor.height_n; r++) {
114             for(int c = 0; c < dst_tensor.width_n; c++) {
115                 for(int ch = 0; ch < dst_tensor.channel_n; ch++) {
116                     tensor_set(&dst_tensor, k, r, c, ch, -1);
117                 }
118             }
119         }
120     }
121
122
123     // Specify source_tensor, dst_tensor, RELU=0
124
125     NVDLA_Uutils_StagePassThroughSDP(&source_tensor, &dst_tensor, RELU);
126
127     // Turn blocks on to complete operation
128     nvdlaregWrite(NVDLA_SDP_D_OP_ENABLE, 0x1);
129     nvdlaregWrite(NVDLA_SDP_RDMA_D_OP_ENABLE, 0x1);
130
131     sleep(5); // Await operation end
132
133     // Verify tensors are the same
134
135
136     for(int k = 0; k < dst_tensor.batch_n; k++) {
137         for(int r = 0; r < dst_tensor.height_n; r++) {
138             for(int c = 0; c < dst_tensor.width_n; c++) {
139                 for(int ch = 0; ch < dst_tensor.channel_n; ch++) {
140                     int a = tensor_get(&source_tensor, k, r, c, ch);
141 #if RELU
142                     if(a < 0) a = 0;
143 #endif

```

```

144         int b = tensor_get(&dst_tensor, k, r, c, ch);
145         if(a != b) {
146             xil_printf("Expected %x, Was %x\r\n", a, b);
147         }
148     }
149 }
150 }
151 }
152
153 return 0;
154 }

```

Listing A.4: Stage SDP passthrough test in NVDLA function.

A.5 Stage Direct Convolution NVDLA

```

1  /*
2  * Program all registers related to performing a
3  * direct convolution in NVDLA given input set of parameters
4  */
5  void NVDLA_Utils_StageDirectConvolution(Tensor *feature, Tensor *weight,
6      Tensor *dest, int strideX, int strideY,
7      int leftPad, int rightPad, int bottomPad, int topPad, int padValue, int
8      dilation, int d_release, int weight_banks, int feat_banks) {
9
10     // Default config the accelerator
11
12     // CDMA
13     nvdlaregWrite(NVDLA_CDMA_S_STATUS, 0);
14     nvdlaregWrite(NVDLA_CDMA_S_POINTER, 0);
15     nvdlaregWrite(NVDLA_CDMA_S_ARBITER, 0x10001); // Default arbiter
16     nvdlaregWrite(NVDLA_CDMA_D_MISC_CFG, 0); // No special misc configurations
17     nvdlaregWrite(NVDLA_CDMA_D_DATAIN_FORMAT, 0); // No special pixel
18     formatting, using Direct Convolution
19     nvdlaregWrite(NVDLA_CDMA_D_DAIN_RAM_TYPE, 0x1); // Default ram type
20     nvdlaregWrite(NVDLA_CDMA_D_WEIGHT_RAM_TYPE, 0x1); // Default ram type
21     nvdlaregWrite(NVDLA_CDMA_D_WEIGHT_FORMAT, 0); // Data is not compressed
22     nvdlaregWrite(NVDLA_CDMA_D_MEAN_FORMAT, 0); // Mean registers not used
23     nvdlaregWrite(NVDLA_CDMA_D_CVT_CFG, 0); // Disable input converter
24     nvdlaregWrite(NVDLA_CDMA_D_DAIN_MAP, 0x10001); // Input data is surface
25     and line packed
26     nvdlaregWrite(NVDLA_CDMA_D_CVT_OFFSET, 0);
27     nvdlaregWrite(NVDLA_CDMA_D_CVT_SCALE, 0);
28     nvdlaregWrite(NVDLA_CDMA_D_WMB_BYTES, 0);
29
30     // CSC
31     nvdlaregWrite(NVDLA_CSC_S_STATUS, 0);
32     nvdlaregWrite(NVDLA_CSC_S_POINTER, 0);
33     nvdlaregWrite(NVDLA_CSC_D_MISC_CFG, 0);

```



```

30  nvdlaregWrite(NVDLA_CSC_D_DATAIN_FORMAT, 0);
31  nvdlaregWrite(NVDLA_CSC_D_POST_Y_EXTENSION, 0);
32  nvdlaregWrite(NVDLA_CSC_D_WEIGHT_FORMAT, 0); // Data is not compressed
33  nvdlaregWrite(NVDLA_CSC_D_WMB_BYTES, 0);
34
35  // CMAC
36  nvdlaregWrite(NVDLA_CMAC_A_S_STATUS, 0);
37  nvdlaregWrite(NVDLA_CMAC_B_S_STATUS, 0);
38  nvdlaregWrite(NVDLA_CMAC_A_S_POINTER, 0);
39  nvdlaregWrite(NVDLA_CMAC_B_S_POINTER, 0);
40  nvdlaregWrite(NVDLA_CMAC_A_D_MISC_CFG, 0);
41  nvdlaregWrite(NVDLA_CMAC_B_D_MISC_CFG, 0);
42
43  // CACC
44  nvdlaregWrite(NVDLA_CACC_S_STATUS, 0);
45  nvdlaregWrite(NVDLA_CACC_S_POINTER, 0);
46  nvdlaregWrite(NVDLA_CACC_D_MISC_CFG, 0);
47  nvdlaregWrite(NVDLA_CACC_D_CLIP_CFG, 0);
48  nvdlaregWrite(NVDLA_CACC_D_OUT_SATURATION, 0);
49  // nvdlaregWrite(NVDLA_CACC_D_DATAOUT_MAP, 0x10001); // Output data is
surface and line packed
50  nvdlaregWrite(NVDLA_CACC_D_DATAOUT_MAP, 0); // Output data is surface and
line packed
51
52
53  // SDP_RDMA NOT REQUIRED IN CONVOLUTION OPERATION
54
55  // SDP
56  nvdlaregWrite(NVDLA_SDP_S_STATUS, 0);
57  nvdlaregWrite(NVDLA_SDP_S_POINTER, 0);
58  nvdlaregWrite(NVDLA_SDP_D_FEATURE_MODE_CFG, 1); // Read from conv core,
set NAN to 0
59  nvdlaregWrite(NVDLA_SDP_D_DP_BS_CFG, SDP_BS_CFG_BYPASS); // Disable BS
60  nvdlaregWrite(NVDLA_SDP_D_DP_BN_CFG, SDP_BN_CFG_BYPASS); // Disable BN
61  nvdlaregWrite(NVDLA_SDP_D_DP_EW_CFG, SDP_EW_CFG_BYPASS); // Disable EW
62  nvdlaregWrite(NVDLA_SDP_D_DST_DMA_CFG, 1);
63  nvdlaregWrite(NVDLA_SDP_D_DATA_FORMAT, 0);
64  nvdlaregWrite(NVDLA_SDP_D_CVT_OFFSET, 0);
65  nvdlaregWrite(NVDLA_SDP_D_STATUS, 0);
66  nvdlaregWrite(NVDLA_SDP_D_CVT_SHIFT, 0);
67  nvdlaregWrite(NVDLA_SDP_D_CVT_SCALE, 1);
68
69
70  /*
71  * == Point accelerator to input and output tensor addresses ==
72  */
73
74  // Address of feature data -> DAIN in NVDLA (Data IN)
75  uint64_t feature_addr = (uint64_t)feature->array;
76
77  // Address of weight data -> WEIGHT in NVDLA

```

```

78     uint64_t weight_addr = (uint64_t)weight->array;
79
80     // Address of output data
81     uint64_t dest_addr = (uint64_t)dest->array;
82
83     nvdlaregWrite(NVDLA_CDMA_D_DAIN_ADDR_LOW_0, feature_addr & 0xFFFFFFFF);
84     nvdlaregWrite(NVDLA_CDMA_D_DAIN_ADDR_HIGH_0, (feature_addr >> 32) & 0
85     xFFFFFFFF);
86
87     nvdlaregWrite(NVDLA_CDMA_D_WEIGHT_ADDR_LOW, weight_addr & 0xFFFFFFFF);
88     nvdlaregWrite(NVDLA_CDMA_D_WEIGHT_ADDR_HIGH, (weight_addr >> 32) & 0
89     xFFFFFFFF);
90
91     nvdlaregWrite(NVDLA_SDP_D_DST_BASE_ADDR_LOW, dest_addr & 0xFFFFFFFF);
92     nvdlaregWrite(NVDLA_SDP_D_DST_BASE_ADDR_HIGH, (dest_addr >> 32) & 0
93     xFFFFFFFF);
94
95     /*
96     * == Pass in input tensor shapes ==
97     */
98
99     // Feature tensor
100     uint8_t feature_datain_size0 = ((feature->height_n-1) << 16) | (feature->
101     width_n-1);
102     uint8_t feature_datain_size1 = feature->channel_n-1;
103
104     nvdlaregWrite(NVDLA_CDMA_D_DATAIN_SIZE_0, feature_datain_size0);
105     nvdlaregWrite(NVDLA_CDMA_D_DATAIN_SIZE_EXT_0, feature_datain_size0);
106     nvdlaregWrite(NVDLA_CDMA_D_DATAIN_SIZE_1, feature_datain_size1);
107     nvdlaregWrite(NVDLA_CDMA_D_BATCH_NUMBER, feature->batch_n-1);
108
109     nvdlaregWrite(NVDLA_CSC_D_DATAIN_SIZE_EXT_0, feature_datain_size0);
110     nvdlaregWrite(NVDLA_CSC_D_DATAIN_SIZE_EXT_1, feature_datain_size1);
111     nvdlaregWrite(NVDLA_CSC_D_BATCH_NUMBER, feature->batch_n-1);
112     nvdlaregWrite(NVDLA_CACC_D_BATCH_NUMBER, feature->batch_n-1);
113
114     nvdlaregWrite(NVDLA_CDMA_D_SURF_STRIDE, (feature->height_n*feature->
115     width_n*ATOMIC_C));
116     nvdlaregWrite(NVDLA_CDMA_D_LINE_STRIDE, (feature->width_n*ATOMIC_C));
117     nvdlaregWrite(NVDLA_CDMA_D_BATCH_STRIDE, (feature->height_n*feature->
118     width_n*feature->channel_n*ATOMIC_C));
119
120     uint8_t cbuf_entry_per_slice = feature->width_n * Xil_Ceil((float)feature
121     ->channel_n / ATOMIC_C);
122     uint8_t banks_per_data = feat_banks;
123     uint8_t banks_per_weight = weight_banks;

```

```

122     nvdlaregWrite(NVDLA_CSC_D_BANK,  banks_per_data | (banks_per_weight << 16)
123 );
124     nvdlaregWrite(NVDLA_CDMA_D_BANK,  banks_per_data | (banks_per_weight <<
125 16));
126
127     nvdlaregWrite(NVDLA_CDMA_D_ENTRY_PER_SLICE,  cbuf_entry_per_slice-1);
128     nvdlaregWrite(NVDLA_CSC_D_ENTRY_PER_SLICE,  cbuf_entry_per_slice-1);
129
130     nvdlaregWrite(NVDLA_CSC_D_RELEASE,  d_release);
131     nvdlaregWrite(NVDLA_CDMA_D_FETCH_GRAIN,  0);
132
133     // Weight tensor
134     // Num of bytes for one kernel batch
135     nvdlaregWrite(NVDLA_CDMA_D_WEIGHT_SIZE_0,  (weight->width_n * weight->
136 height_n * weight->channel_n)-1);
137     // Num of batches
138     nvdlaregWrite(NVDLA_CDMA_D_WEIGHT_SIZE_1,  (weight->batch_n-1));
139     nvdlaregWrite(NVDLA_CDMA_D_WEIGHT_BYTES,  (weight->batch_n * weight->
140 width_n * weight->height_n * weight->channel_n));
141
142     nvdlaregWrite(NVDLA_CSC_D_WEIGHT_SIZE_EXT_0,  (weight->width_n-1) | ((
143 weight->height_n-1) << 16));
144     nvdlaregWrite(NVDLA_CSC_D_WEIGHT_SIZE_EXT_1,  (weight->channel_n-1) | ((
145 weight->batch_n-1) << 16));
146     nvdlaregWrite(NVDLA_CSC_D_WEIGHT_BYTES,  (weight->batch_n * weight->width_n
147 * weight->height_n * weight->channel_n));
148
149     /*
150     * == Convolution Parameters ==
151     */
152
153     nvdlaregWrite(NVDLA_CDMA_D_CONV_STRIDE,  (strideX-1) | ((strideY-1) << 16)
154 );
155     nvdlaregWrite(NVDLA_CSC_D_CONV_STRIDE_EXT,  (strideX-1) | ((strideY-1) <<
156 16));
157
158     // Padding format { 2'b0, pad_bottom, 3'b0, pad_top, 2'b0, pad_right, 3'b0
159 , pad_left };
160     // bits - pb:6, pl:5, pr:6, pt:5
161     nvdlaregWrite(NVDLA_CDMA_D_ZERO_PADDING,  (leftPad) | (rightPad << 8) | (
162 topPad << 16) | (bottomPad << 24));
163     nvdlaregWrite(NVDLA_CDMA_D_ZERO_PADDING_VALUE,  padValue);
164
165     // Output tensor shape
166     dest->batch_n = 1;
167     dest->height_n = 1 + (topPad + feature->height_n + bottomPad - weight->
168 height_n) / strideY;

```

```

160     dest->width_n = 1 + (leftPad + feature->width_n + rightPad - weight->
width_n) / strideX;
161     dest->channel_n = weight->batch_n;
162
163     nvdlaregWrite(NVDLA_CSC_D_DATAOUT_SIZE_0, (dest->width_n-1) | ((dest->
height_n-1) << 16));
164     nvdlaregWrite(NVDLA_CSC_D_DATAOUT_SIZE_1, (dest->channel_n-1));
165     nvdlaregWrite(NVDLA_SDP_D_DATA_CUBE_WIDTH, (dest->width_n-1));
166     nvdlaregWrite(NVDLA_SDP_D_DATA_CUBE_CHANNEL, (dest->channel_n-1));
167     nvdlaregWrite(NVDLA_SDP_D_DATA_CUBE_HEIGHT, (dest->height_n-1));
168
169     nvdlaregWrite(NVDLA_CACC_D_DATAOUT_SIZE_0, (dest->width_n-1) | ((dest->
height_n-1) << 16));
170     nvdlaregWrite(NVDLA_CACC_D_DATAOUT_SIZE_1, (dest->channel_n-1));
171     nvdlaregWrite(NVDLA_CSC_D_ATOMICS, (dest->width_n * dest->height_n) -1 );
172     nvdlaregWrite(NVDLA_CSC_D_DILATION_EXT, dilation);
173     nvdlaregWrite(NVDLA_CSC_D_ZERO_PADDING, (leftPad) | (topPad << 16));
174     nvdlaregWrite(NVDLA_CSC_D_ZERO_PADDING_VALUE, padValue);
175
176     nvdlaregWrite(NVDLA_SDP_D_DST_SURFACE_STRIDE, (dest->height_n*dest->
width_n*ATOMIC_C));
177     nvdlaregWrite(NVDLA_SDP_D_DST_LINE_STRIDE, (dest->width_n*ATOMIC_C));
178     nvdlaregWrite(NVDLA_CACC_D_SURF_STRIDE, (dest->height_n*dest->width_n*
ATOMIC_C));
179     nvdlaregWrite(NVDLA_CACC_D_LINE_STRIDE, (dest->width_n*ATOMIC_C));
180
181 }

```

Listing A.5: Stage direct convolution in NVDLA function.

A.6 Stage Direct Convolution NVDLA

```

1 void generate_timings(Tensor *source, Tensor *weight) {
2     Tensor dest_tensor_nvdlareg;
3     Tensor dest_tensor_arm;
4
5     for(int k = 0; k < source->batch_n; k++) {
6         for(int r = 0; r < source->height_n; r++) {
7             for(int c = 0; c < source->width_n; c++) {
8                 for(int ch = 0; ch < source->channel_n; ch++) {
9                     tensor_set(source, k, r, c, ch, 1);
10                }
11            }
12        }
13    }
14
15    for(int k = 0; k < weight->batch_n; k++) {
16        for(int r = 0; r < weight->height_n; r++) {
17            for(int c = 0; c < weight->width_n; c++) {

```

```

18         for(int ch = 0; ch < weight->channel_n; ch++) {
19             tensor_set(weight, k, r, c, ch, 1);
20         }
21     }
22 }
23 }
24
25 memset(dest_tensor_nvdlA.array, -1, 1000);
26 memset(dest_tensor_arm.array, -1, 1000);
27
28
29
30
31 float test_time;
32 char buffer[100];
33
34 XTime gbl_time_before_test;
35 XTime *p_gbl_time_before_test = &gbl_time_before_test;
36 XTime gbl_time_after_test;
37 XTime *p_gbl_time_after_test = &gbl_time_after_test;
38
39
40 XTime_GetTime(p_gbl_time_before_test);
41 tensor_convolution(&dest_tensor_arm, source, weight, 1, 1, 0, 0, 0, 0);
42 XTime_GetTime(p_gbl_time_after_test);
43 test_time = (float) (gbl_time_after_test - gbl_time_before_test)/(
COUNTS_PER_SECOND);
44 sprintf(buffer, "ARM elapsed: %.8f secs\r\n", test_time);
45 xil_printf(buffer);
46
47 // Test_Utils_ReshapeTensorNVDLA (&source_tensor);
48 // Test_Utils_ReshapeTensorNVDLA (&weight_tensor);
49
50
51 Xil_DCacheDisable();
52
53 NVDLA_Utils_StageDirectConvolution(source, weight, &dest_tensor_nvdlA, 1,
1, 0, 0, 0, 0, 0,
54 0, 0, 15, 15);
55
56
57 XTime_GetTime(p_gbl_time_before_test);
58 NVDLA_Utils_AwaitDirectConvolution();
59 XTime_GetTime(p_gbl_time_after_test);
60 test_time = (float) (gbl_time_after_test - gbl_time_before_test)/(
COUNTS_PER_SECOND);
61
62 sprintf(buffer, "NVDLA elapsed: %.8f secs\r\n", test_time);
63 xil_printf(buffer);
64
65

```

```

66 Test_Utils_ReshapeNVDLATensor(&dest_tensor_nvdlA);
67 Xil_DCacheEnable();
68
69 // xil_printf("ARM core results:\n");
70 // display_tensor_attrs(&dest_tensor_arm);
71 // display_tensor_array(&dest_tensor_arm, "dest_tensor_arm");
72
73 // xil_printf("NVDLA results:\n");
74 // display_tensor_attrs(&dest_tensor_nvdlA);
75 // display_tensor_array(&dest_tensor_nvdlA, "dest_tensor_nvdlA");
76
77 int verified = 1;
78 for(int k = 0; k < dest_tensor_arm.batch_n; k++) {
79     for(int r = 0; r < dest_tensor_arm.height_n; r++) {
80         for(int c = 0; c < dest_tensor_arm.width_n; c++) {
81             for(int ch = 0; ch < dest_tensor_arm.channel_n; ch++) {
82                 if(tensor_get(&dest_tensor_arm, k, r, c, ch) != tensor_get
83 (&dest_tensor_nvdlA, k, r, c, ch)) {
84                     // xil_printf("%s[%d][%d][%d][%d] = {%d} ", "
dest_tensor_arm", k, r, c, ch, tensor_get(&dest_tensor_arm, k, r, c, ch));
85                     // xil_printf("%s[%d][%d][%d][%d] = {%d}\n", "
dest_tensor_nvdlA", k, r, c, ch, tensor_get(&dest_tensor_nvdlA, k, r, c,
ch));
86
87                     verified = 0;
88                     break;
89                 }
90             }
91         }
92     }
93
94     xil_printf((verified)? "Verified\r\n" : "Invalid\r\n");
95     return;
96 }
97
98 int dc_test_tensor()
99 {
100
101     Tensor source_tensor = {
102         .batch_n = 1,
103         .height_n = 1,
104         .width_n = 32,
105         .channel_n = 16,
106     };
107
108     Tensor weight_tensor = {
109         .batch_n = 8,
110         .height_n = 1,
111         .width_n = 32,
112         .channel_n = 16

```

```
113     };  
114  
115  
116     // for(int i = 8; i < 64; i++) {  
117         generate_timings(&source_tensor, &weight_tensor);  
118         // source_tensor.width_n += 1;  
119     // }  
120  
121  
122     return 1;  
123 }
```

Listing A.6: Testing and timing direct convolution with tensors.